

# Hardware Security

Author  
Hardware Security Survey

**Abstract**—This survey reviews defense mechanisms for Arm and RISC-V confidential computing, confidential-computing network and I/O data paths, and ISA-level hardware design defenses. The central observation is that modern hardware security cannot be evaluated by CPU isolation alone. A confidential workload is protected only when execution domains, physical-memory ownership, device assignment, DMA translation, interrupt routing, link protection, and attestation evidence compose into a coherent platform boundary. The survey therefore connects Arm CCA, RISC-V CoVE/AP-TEE, trusted I/O protocols, accelerator TEEs, and hardware mechanisms such as MTE, PMP, capability machines, and CFI extensions under a common systems-oriented taxonomy.

**Index Terms**—hardware security, confidential computing, trusted execution environments, Arm CCA, RISC-V, attestation, I/O protection

## I. INTRODUCTION

Hardware security has shifted from isolated processor features toward platform-level protection contracts. Modern systems must coordinate execution domains, memory ownership, device access, interrupt delivery, and attestation evidence across the whole machine. This survey organizes the current literature around that composition problem, using Arm and RISC-V as the main architectural lines.

The scope is deliberately narrower than a general hardware-security SoK. We focus on three defense-oriented lanes. The first lane covers confidential computing mechanisms in Arm and RISC-V systems, including Arm TrustZone and CCA as well as RISC-V enclave lineage, CoVE, AP-TEE, and TVM lifecycle management. The second lane covers network, I/O, and data-path defense in confidential-computing deployments, including SPDM, TDISP, PCIe IDE, IOMMU/IOPMP/SMMU mediation, trusted MSI, SmartNIC/DPU-style offload, and accelerator TEEs. The third lane covers ISA and hardware design defenses that provide defense-in-depth for those platforms: memory tagging, page-table permissions, PMP/ePMP, capability/tagging architectures, hardware CFI, memory-encryption taxonomy, and debug or trace controls.

This scope also defines what the survey does not attempt to cover in the current version. Side channels, physical leakage, fault injection, Rowhammer, and attack-only vulnerability catalogs are important hardware-security topics, but they are not treated as independent research lanes here. They are mentioned only when they affect the threat-model boundary of a confidential-computing mechanism. Similarly, ordinary network-security mechanisms such as firewalls, generic IDS/IPS systems, routing security, TLS, or VPNs are excluded unless they directly protect a confidential workload’s trusted

network endpoint, device assignment path, or secure offload boundary.

The paper is organized as follows. The Arm section explains the transition from TrustZone-style TEEs to CCA and Realm-based protection. The RISC-V section connects earlier enclave systems to CoVE/AP-TEE confidential VMs. The confidential I/O section then treats devices, fabrics, and network data paths as first-class parts of the confidential-computing boundary. Finally, the hardware-design section surveys cross-architecture ISA and microarchitectural defenses that complement those confidential-computing mechanisms.

## II. METHODOLOGY AND EVIDENCE RANKING

This survey separates evidence strength from topical relevance. A source may be important to the taxonomy while still being unsuitable for a mechanism-level claim. We therefore use the following evidence classes when deciding how a reference may support the text:

TABLE I  
EVIDENCE CLASSES AND CITATION RULES

| Class | Evidence type                                                                    | Allowed use in this survey                                                                                                                        |
|-------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| E0    | Official ratified specification, architecture manual, RFC, or standards document | Normative mechanism and status claims, with no experimental claims unless the source provides them.                                               |
| E1    | Peer-reviewed primary research paper                                             | Mechanism, evaluation, and threat-model claims supported by the reviewed paper and its recorded source/PDF status.                                |
| E2    | Survey, SoK, or secondary review                                                 | Taxonomy, terminology, and coverage guidance; original mechanism claims are traced back to primary papers or specifications.                      |
| E3    | Public draft, release candidate, or not-ratified specification                   | Current standardization direction, always labelled as draft, release candidate, or not ratified.                                                  |
| E4    | Vendor, industry, or product documentation                                       | Industry evidence for product behavior or deployment practice, not proof of general security unless independently supported.                      |
| E5    | Metadata-only, gated, blocked-PDF, or HTML-only evidence                         | Existence, source status, or backlog classification only; no substantive mechanism claim unless promoted or paired with stronger public evidence. |

The citation rules are intentionally conservative. First, all mechanism claims prefer E0 or E1 sources. Second, E2 sources such as surveys and SoKs are used as maps of the literature, not as substitutes for original mechanisms. Third, E3 drafts are useful for current architecture direction, but

the text must state their draft or not-ratified status. Fourth, E4 industry documents are treated as deployment or product evidence and kept separate from peer-reviewed system claims. Fifth, E5 sources, including metadata-only `rw...` candidates and member-gated documents, do not support substantive text claims unless a public source supports the same claim. When a source has mixed properties, such as a public metadata page for a gated specification or an HTML-only survey, the more restrictive citation rule controls the claim.

Tables II and III make the strongest sources and the main boundary cases visible in the paper body. In particular, draft RISC-V specifications, vendor documents, surveys, and gated PCI-SIG metadata are listed with their restricted claim roles rather than being treated as ordinary mechanism evidence.

### III. TEE DESIGN SPACE AND EVIDENCE BOUNDARIES

Before comparing Arm CCA, RISC-V CoVE/AP-TEE, and confidential I/O, it is useful to separate the design dimensions that recur across TEE systems. Survey and SoK sources provide the taxonomy for this section, while platform-specific mechanism claims in later sections still return to primary papers and official specifications [1]–[4].

Table IV provides the bridge used by the rest of the survey. Arm CCA and RISC-V CoVE/AP-TEE both address launch, runtime isolation, attestation, and TCB control, but they expose different interfaces and standardization status. Confidential I/O and secure storage extend the same questions beyond CPU execution into DMA, interrupts, links, devices, and persistent or remote state. Keeping these dimensions separate prevents three common overclaims: treating attestation as isolation, treating memory ownership as storage freshness, and treating device identity as complete trusted I/O.

Deployment substrates introduce another boundary that is easy to overstate. **CoFunc** studies confidential serverless functions through split containers, placing a function-oriented microkernel and library OS inside a CVM while leaving Linux container management in an untrusted shadow container outside the CVM [5]. This is useful evidence for confidential container and serverless packaging because it exposes the tension between CVM cold start, resource sharing, TCB size, and function isolation. It is not an Arm CCA mechanism paper and should not be used to justify Realm lifecycle, RMM/RMI semantics, device assignment, or trusted-I/O claims.

### IV. ARM CONFIDENTIAL COMPUTING

Confidential computing in the Arm ecosystem addresses a stronger threat model than classical trusted-execution environments. The central problem is not only how to isolate a secure software stack from ordinary software, but how to protect the confidentiality of a workload even when privileged software still manages scheduling, memory allocation, and devices [6]–[10]. In Arm platforms, this evolution is visible in the transition from TrustZone-based TEEs to the Arm Confidential Compute Architecture (CCA), which extends the protection boundary into physical-memory ownership, device mediation, and attestation. The following sections survey this

transition, covering TrustZone-based systems, Realm-based isolation, hardware enforcement in the memory and I/O path, and the attestation mechanisms needed to make protected execution usable in practice.

#### A. Threat Model and Design Objectives

1) *From Trusted Services to Host-Resilient Confidentiality:* The distinction between TrustZone-era TEEs and Arm CCA is fundamentally a distinction in threat model. In a classical TrustZone deployment, the secure-world firmware and runtime environment are themselves part of the trusted computing base, and the main goal is to isolate security services from the normal world [6], [7]. In the confidential-computing setting addressed by Arm CCA, the protected workload must remain confidential even though a host stack continues to manage cores, memory allocation, and devices [8]–[10]. The architecture therefore aims to separate resource management from data visibility more explicitly than conventional TEE models do.

2) *Core Design Dimensions:* The cited Arm CCA documents and systems papers show that this protection problem decomposes into several recurring design dimensions. The first is execution-domain structure: the architecture needs a protected execution context that is distinct from both ordinary software and the management software responsible for scheduling and provisioning [8]–[10]. The second is memory ownership: physical memory must carry hardware-enforced ownership metadata rather than depending only on hypervisor-maintained bookkeeping [9]–[11]. The third is system integration: I/O, accelerators, interrupts, and attestation all have to reflect the same protection boundary if confidentiality is to survive contact with the whole machine [12]–[16]. These dimensions provide a useful organizing lens for the remainder of the chapter.

#### B. From TrustZone-Based TEEs to Arm CCA

1) *TrustZone as the Classical Arm Isolation Model:* **TrustZone** partitions an Arm system into Secure and Non-secure worlds, with the Non-secure bit propagated through the SoC interconnect and memory system [6], [7]. This model has been widely used to build TEEs for mobile, embedded, and IoT devices, where a relatively small secure-world software stack mediates access to protected resources. Representative systems such as **TrustShadow** show that TrustZone can protect unmodified applications from an untrusted normal-world operating system by relocating sensitive execution into the secure world [17]. At the same time, survey and SoK work shows that TrustZone-assisted TEEs remain exposed to recurring weaknesses in interfaces, implementation choices, and trusted computing base growth [7], [18].

For hardware-security surveys, the importance of TrustZone is twofold. First, it demonstrates that Arm’s security story has long depended on system-wide enforcement rather than on CPU privilege levels alone. Second, it shows the limits of a two-world model once the secure world accumulates more functionality: as more services migrate into the trusted side,

TABLE II  
KEY PLATFORM AND STANDARD SOURCES

| Key                  | Class | Status/version                                                       | Allowed claim in this survey                                                                              |
|----------------------|-------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| arm_cca_spec         | E0    | Arm official specification, 2025                                     | CCA architecture semantics, Realm security state, and architectural protection model.                     |
| arm_rme_spec         | E0    | Arm official RME specification, 2025                                 | RME, GPT/GPC, granule ownership, and physical-address-space semantics.                                    |
| arm_rmm_spec         | E0    | Arm official RMM specification, 2025                                 | RMM lifecycle, RMI/RSI interface roles, and monitor-side CCA state.                                       |
| li2022cca            | E1    | OSDI 2022 primary paper                                              | Arm CCA design and verification claims supported by the paper.                                            |
| arm_smmu_spec        | E0    | Arm SMMUv3 architecture specification                                | DMA translation and system I/O mediation substrate, not complete trusted I/O by itself.                   |
| riscv_privileged     | E0    | RISC-V privileged architecture                                       | Privilege modes, PMP/ePMP/Smepmp, virtualization substrate, and page-table semantics.                     |
| riscv_iommu_2023     | E0    | RISC-V IOMMU v1.0.1 / 20260222 ratified-library release              | Device-side DMA translation substrate for confidential-I/O composition.                                   |
| riscv_aia_2023       | E0    | RISC-V AIA v1.0, ratified June 2023; revised 20250312 clarifications | AIA/IMSIC interrupt substrate and trusted-MSI vocabulary.                                                 |
| riscv_iopmp_2026     | E3    | IOPMP v0.8.2 draft, not ratified                                     | Current I/O protection direction; draft status must remain visible.                                       |
| sahita2023cove       | E3    | CoVE public preprint / proposal                                      | CoVE motivation and reference-architecture direction, not ratified ABI detail.                            |
| riscv_ap_tee_2024    | E3    | Draft/not ratified: AP-TEE v0.7 / RC2                                | RISC-V TVM/TSM lifecycle and ABI direction, labelled as draft.                                            |
| riscv_cove_io_2026   | E3    | Draft/not ratified: CoVE-IO v0.3.0                                   | Trusted-I/O architecture direction for RISC-V TVMs, labelled as draft.                                    |
| dmtf_spdm_2025       | E0    | DMTF DSP0274 v1.4.0, selected as this survey's SPDm snapshot         | SPDM identity, measurement, negotiation, and key-exchange semantics.                                      |
| pcisig_tdisp_2022    | E5    | Public ECN metadata; member-gated PDF                                | TDISP existence, affected-specification metadata, and high-level device-interface role only.              |
| pcisig_tdisp_xt_2025 | E5    | Public PCIe 6.x ECN metadata; member-gated download                  | TDISP XT public existence and affected IDE/ATS/TDISP metadata only; no gated XT/TE normative-rule claims. |
| eat_rfc              | E0    | RFC 9711, Standards Track                                            | EAT claim/token encoding for attestation evidence.                                                        |
| rats_rfc             | E0    | RFC 9334, Standards Track                                            | RATS attester, verifier, relying-party, evidence, and appraisal roles.                                    |

the TCB grows and the isolation boundary becomes harder to reason about.

2) *Arm CCA and the Realm Model*: **Arm CCA** extends this design through the **Realm Management Extension (RME)**, introducing a model in which confidential workloads execute inside **Realms** that are intended to remain opaque to the host software stack [8]–[10]. The associated **Realm Management Monitor (RMM)** manages Realm creation, execution, and teardown [15], [19]. Compared with TrustZone, the key change is not merely the addition of another execution environment; it is the explicit separation between resource management and data ownership. The host still provisions CPUs, memory, and devices, but it is not meant to inspect Realm memory or register state as part of normal management.

This change has already motivated systems work above the raw architecture. **SHELTER** uses Arm CCA to extend isolation into user space [20]. **RContainer** treats CCA hardware primitives as a basis for container isolation [21]. **virtCCA** studies how CCA-style protection can be virtualized in conjunction with existing Arm trust mechanisms [22]. Taken together, these works show that Arm CCA is being treated not only as an architectural feature set, but also as a substrate for higher-level confidential-computing software stacks.

Table V fixes the mechanism vocabulary used in the rest of this survey. The main distinction is between *management authority* and *data authority*. CCA still allows the host to allocate CPUs, schedule execution, and request lifecycle transitions through RMI, but GPT/GPC checks, RMM state, PAS/RIPAS

transitions, and Realm-facing RSI calls constrain what that management software can observe or mutate [9]–[11], [15]. This table also prevents a common classification error: CCA memory ownership metadata should not be described as memory encryption, and RMM lifecycle logic should not be collapsed into ordinary page-table policy.

### C. Hardware Enforcement in the Memory and I/O Path

1) *Granule Ownership and Physical-Memory Checks*: The stronger isolation model of Arm CCA depends on hardware checks in the physical-memory path. The **Granule Protection Table (GPT)** records the ownership and security state of memory granules, while the **Granule Protection Check (GPC)** validates physical accesses against that metadata [9]–[11]. This is a significant departure from designs in which isolation is represented only by software-maintained page tables or a small number of coarse protected regions. In the CCA model, physical-memory ownership becomes a hardware-checked property that constrains even privileged software.

This ownership model is also what makes CCA suitable for dynamic workloads. Confidential workloads may be created, destroyed, resized, or migrated, and their protected memory footprint may change over time. Recent work such as **SHELTER**, **NanoZone**, and *More Granular, Less Trust* exploits this finer-grained ownership model to study user-space and intra-process isolation under an untrusted manager [20], [23], [24]. The common theme across these systems is that the architecture is no longer treating protected memory as a

TABLE III  
KEY SURVEY, VENDOR, AND BOUNDARY SOURCES

| Key                                                    | Class | Status/version                                                                    | Allowed claim in this survey                                                                                                        |
|--------------------------------------------------------|-------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| nvidia_bluefield<br>operation_2025                     | E4    | NVIDIA BlueField public documentation                                             | DPU OP-TEE/TPM building-block evidence only, not complete confidential networking proof.                                            |
| boubakri2025riscvtee                                   | E2    | Peer-reviewed Electronics 2025 survey; MDPI HTML and static-resource PDF verified | RISC-V TEE taxonomy support only; mechanism claims return to primary papers or specs.                                               |
| schneider2022soktee                                    | E2    | Hardware-supported TEE SoK / survey                                               | Cross-platform TEE design-space taxonomy and terminology.                                                                           |
| li2024sokteechoices                                    | E2    | ASIA CCS 2024 SoK                                                                 | TEE runtime design-choice and pitfall taxonomy, not platform-specific mechanism proof.                                              |
| sok-tee                                                | E2    | Accelerator TEE SoK                                                               | Accelerator/device-TEE taxonomy; not general CCA or CoVE mechanism evidence.                                                        |
| perkins2024socsok                                      | E2    | SoC-FPGA TEE SoK                                                                  | SoC-FPGA accelerator-TEE coverage and gap map; primary mechanism claims return to original systems.                                 |
| abera2016cflat<br>dessouky2017lofat<br>nunes2019vrased | E1    | Peer-reviewed control-flow and verified remote-attestation lineage                | Runtime/control-flow and verified RA mechanism evidence; not generic RATS/EAT token semantics or confidential-VM evidence profiles. |
| ferraiuolo2017komodo                                   | E1    | SOSP 2017 verified enclave-monitor system                                         | Verified TEE monitor lineage and TrustZone prototype evidence; not Arm CCA or RISC-V CoVE/AP-TEE evidence.                          |
| fan2025xputee                                          | E5    | ACM TOCS metadata; local PDF unavailable                                          | Recent CPU/GPU heterogeneous-TEE baseline at abstract level only; no strong mechanism or performance claim.                         |
| xia2021sgxfpga                                         | E5    | DAC metadata; local PDF unavailable                                               | CPU-FPGA TEE lineage marker only; no strong mechanism claim without the paper PDF.                                                  |
| feng2024siopmp                                         | E1    | ASPLOS 2024 primary paper                                                         | sIOPMP mechanism and evaluation claims for scalable I/O protection.                                                                 |

TABLE IV  
TEE DESIGN-SPACE DIMENSIONS AND EVIDENCE BOUNDARIES

| Dimension                       | Survey question                                                                                | Typical mechanisms                                                                         | Evidence boundary                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Launch and admission            | How does a protected domain enter a known initial state before receiving secrets?              | Secure boot, measured launch, Realm/TVM creation, monitor-mediated acceptance              | Use SoK sources for vocabulary; use CCA, AP-TEE, and platform specs for mechanism details.                       |
| Runtime isolation               | Which agents can observe or mutate execution state and protected memory during operation?      | World/domain separation, PMP, GPT/GPC, page lifecycle, enclave monitor policy              | Distinguish process, enclave, Realm, and confidential-VM threat models.                                          |
| Trusted I/O                     | How can a protected workload use devices without handing data authority back to the host?      | SMMU/IOMMU/IOPMP, SPDMM, TDISP, PCIe IDE, CoVE-IO, device assignment                       | Treat drafts, gated specifications, and vendor documents under their restricted evidence classes.                |
| Secure storage and persistence  | What protects data once it leaves CPU-local protected memory or survives reboot?               | Sealing, RPMB/TPM, encrypted storage, freshness metadata, remote-storage integrity         | Do not infer storage freshness or rollback protection from execution isolation alone.                            |
| Attestation and verifier policy | What evidence lets a remote party decide whether to release keys or trust a service?           | RATS roles, EAT tokens, platform evidence, device evidence, endpoint binding               | RFCs define roles and token formats; platform claims need platform-specific evidence.                            |
| TCB and excluded attacks        | Which hardware, firmware, runtime, host, and device components remain trusted or out of scope? | Monitor/RMM/TSM, firmware roots, device managers, verifier policy, side-channel exclusions | Keep side-channel and physical attacks as threat-boundary statements unless a mechanism directly addresses them. |

single static partition; instead, it supports repeated ownership transitions while preserving hardware-enforced separation.

2) *Lifecycle Management and Ownership Transitions*: The granule model is closely tied to lifecycle management. The **RMM** and the Arm CCA software model treat Realm creation, memory delegation, execution, and teardown as explicit state transitions rather than as ad hoc software conventions [11], [15]. This matters because confidential-computing platforms must repeatedly move memory between protection states while preserving consistency between CPU execution, physical-memory ownership, and the host’s remaining management authority. The design and verification work of Li et al. [8] and later verification-oriented work on the **RMM** [19] both reinforce the point that lifecycle state management is part of

the security argument, not merely an implementation detail.

3) *Securing Devices, Accelerators, and Memory Fabrics*: CPU-side isolation is insufficient if **DMA**-capable devices can still reach protected memory directly. Arm therefore extends the confidentiality story into the I/O path through **SMMUv3**-based mediation and realm-aware device assignment [12]–[14]. This matters because many confidential workloads depend on accelerators, storage engines, network adapters, or packet-processing units. If device access is not mediated under the same protection regime as CPU access, the device path becomes the obvious route around the confidentiality boundary.

Recent systems research makes this point concrete. **ACAI** studies how accelerator execution can be protected using Arm

TABLE V  
ARM CCA MECHANISM MAP

| Mechanism                 | Role in CCA                                                                        | Security Property                                                                 | Survey Boundary                                                         |
|---------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Realm security state      | Separates Realm execution from Non-secure, Secure, and Root worlds                 | Host cannot directly inspect Realm execution state under the CCA model            | Confidential VM / Realm isolation, not a generic process sandbox        |
| RMM                       | Mediates Realm creation, entry, exit, destruction, and memory lifecycle operations | Keeps host resource management outside the Realm data boundary                    | RMM is in the CCA TCB; correctness matters as much as hardware support  |
| GPT/GPC                   | Records and checks physical granule ownership and security state                   | Blocks accesses inconsistent with the assigned physical-address space             | Access-control and ownership metadata, not by itself memory encryption  |
| PAS and RIPAS             | Represent physical-address-space assignment and Realm IPA state                    | Makes delegation, acceptance, destruction, and sharing explicit state transitions | Needed to reason about stale mappings, sharing, and reclaim races       |
| RMI                       | Host-to-RMM management interface for Realm and granule lifecycle                   | Lets an untrusted host manage resources through constrained operations            | Management authority is preserved but data visibility is removed        |
| RSI                       | Realm service interface used by Realm software to request CCA services             | Gives Realm software a controlled way to interact with the trusted CCA layer      | Must be distinguished from host-facing RMI calls                        |
| Attestation evidence      | Reports platform, RMM, and initial Realm state to a verifier                       | Enables key release and remote trust decisions                                    | Evidence usefulness depends on verifier policy and measured claims      |
| SMMU and device mediation | Constrains DMA and device-visible memory for Realm-related I/O                     | Prevents CPU-only isolation from being bypassed by devices                        | Requires confidential I/O mechanisms beyond base Realm memory ownership |

CCA mechanisms [13]. **CAGE** extends this line by adapting GPU and FPGA accelerator workflows to Realm-style execution with shadow tasks and GPC/GPT-backed checks [25]. **PORTAL** addresses a neighboring device-access problem for mobile SoCs: it uses CCA memory isolation to create a protected plaintext device-access region instead of encrypting every transfer through an untrusted shared buffer [26]. **Devlore** focuses on device-interrupt protection for confidential VMs, highlighting that interrupt delivery and device ownership are part of the same protection problem [14]. Earlier Arm GPU TEE work such as **StrongBox** is useful as historical context because it shows how TrustZone and Stage-2 translation were used to protect unified-memory GPU tasks before CCA-style Realm mechanisms became the main confidential-computing target [27]. Related work on **CXL**, **PCIe IDE**, heterogeneous memory descriptions, and remote paging is background substrate: it shows that disaggregated memory and high-speed interconnects affect where protected data resides and which agents can observe it, but those sources are not themselves Arm CCA mechanisms or complete trusted-I/O evidence [28]–[33]. For confidential workloads, the integrity of the protection boundary therefore depends on the fabric and device path as much as on the CPU core.

#### D. Software Stacks and Deployment Models

1) *User-Space, Containers, and Virtualized Realms*: One reason Arm CCA has attracted attention is that it can support several deployment models above the raw hardware interface. **SHELTER** explores user-space isolation with Arm CCA hardware primitives [20]. **RContainer** extends the same substrate toward confidential containers [21]. **virtCCA** studies virtualized CCA deployments that combine Realm-style

protection with existing Arm trust infrastructure [22]. **Aster** brings the same question into the Android ecosystem by mapping Android Virtualization Framework protected VMs to CCA Realms and identifying the boot, attestation, rollback, and memory-protection changes needed for mobile protected workloads [34]. These works are useful in a survey because they show that CCA is not restricted to one software packaging model; rather, it is being used as a common basis for protected processes, containers, virtualized workloads, and mobile protected VMs.

Table VI separates these systems by the software object they protect rather than by paper name. This matters because a process compartment, container, nested CCA stack, inter-CVM channel, and device path stress different parts of the architecture. The table also keeps evidence roles visible: peer-reviewed systems can support their own mechanisms and measurements, while arXiv or pre-standardization work is treated as emerging research evidence rather than as proof of mature deployment.

**CPC** adds a separate lifecycle dimension to this deployment taxonomy: CVM maintenance. Instead of giving an untrusted host direct access to guest private state or moving maintenance modules into firmware, CPC lets the host trigger tenant-trusted maintenance procedures inside the CVM through dedicated vCPU scheduling semantics [35]. This makes live migration, snapshotting, and resource reclamation part of the confidential-computing management problem. The paper’s AMD SEV-SNP prototype provides real-machine performance evidence, while the Arm CCA prototype is simulator-based; therefore this survey uses CPC as peer-reviewed evidence for the maintenance/lifecycle gap, not as an Arm official interface or production CCA migration standard.

TABLE VI  
ARM CCA DEPLOYMENT TAXONOMY

| Category                      | Representative work                 | Protected object                                        | CCA mechanism used                                                                       | Evidence role                   | Main limitation                                                                        |
|-------------------------------|-------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------------------|
| User-space isolation          | SHELTER                             | Process or application compartment                      | Realm/GPT/GPC-backed user-space isolation                                                | E1 primary paper                | Not a general VM or container deployment.                                              |
| Container isolation           | RContainer                          | Container workload                                      | CCA primitives beneath container packaging                                               | E1 primary paper                | Deployment maturity and orchestration integration remain open.                         |
| Virtualized CCA               | virtCCA                             | Virtualized Realm/CCA stack                             | CCA combined with existing Arm trust infrastructure                                      | E3 arXiv/preprint evidence      | Nested trust and virtualization complexity.                                            |
| Mobile protected VM           | Aster                               | Android AVF protected VM                                | Realm-world pVM execution with DICE, policy, rollback, and memory-protection integration | E1 emerging mobile CCA evidence | Evaluated with emulator/Armv8.2 approximation rather than native CCA Android hardware. |
| Intra-process isolation       | NanoZone; More Granular, Less Trust | Fine-grained compartment inside a process               | Granule ownership and Realm-style memory protection                                      | E3 plus E1                      | Granularity and draft/preprint status vary by work.                                    |
| CVM maintenance               | CPC                                 | Migration, snapshot, and resource reclamation procedure | Guest-side confidential procedure plus CPTI-style intra-CVM isolation                    | E1 primary paper                | Arm CCA evidence uses official simulator rather than production hardware.              |
| Inter-CVM sharing             | CAEC                                | Communication between confidential VMs                  | Attestation plus controlled sharing over CCA substrate                                   | E3 arXiv evidence               | Not an Arm official interface; sharing policy remains research.                        |
| Device and accelerator access | PORTAL; CAGE; Devlore               | SoC device, accelerator workflow, or interrupt path     | SMMU, GPT/GPC, GPC-backed checks, and Realm-aware mediation                              | E1 plus E3                      | Device identity and complete trusted-I/O lifecycle are still incomplete.               |

**Aster** is treated with a similar boundary discipline. It is useful peer-reviewed evidence that Arm CCA can be studied as an Android AVF backend, including pVM launch policy, DICE-compatible attestation, rollback protection, and per-pVM memory protection [34], [36]. It is not an Arm official mobile profile, and its evaluation is constrained by the absence of native CCA-enabled Android hardware. The survey therefore uses it as emerging mobile deployment evidence rather than as production Android CCA availability.

2) *Fine-Grained Isolation Above the Base Architecture*: Later systems work also shows that Arm CCA is being pushed toward finer-grained protection goals than those envisioned by a simple confidential-VM model. **NanoZone** focuses on scalable and efficient memory protection for Arm CCA [23]. *More Granular, Less Trust* studies intra-process isolation under an untrusted management environment [24]. Together with **SHELTER**, these works indicate that once granule ownership and Realm mechanisms exist, researchers naturally try to repurpose them for user-space compartmentalization and finer-grained software isolation. This is an important trend because it suggests that Arm CCA is evolving from a narrowly server-oriented confidential-computing mechanism into a more general substrate for trusted isolation.

#### E. Attestation and Trust Establishment

1) *Attestation for Realms*: Isolation alone does not let a remote party decide whether to provision secrets to a workload. Arm CCA therefore couples protected execution with attestation evidence describing the platform state, the management monitor, and the initial Realm state [9], [15]. The **RATS** architecture defines the attester, verifier, relying-party,

evidence, and appraisal roles, while the **Entity Attestation Token (EAT)** standard provides a general token format for carrying attestation claims in a machine-consumable form [16], [37]. **DICE** supplies lower-level boot identity vocabulary by deriving a Compound Device Identifier from a Unique Device Secret and first mutable code measurement before mutable code runs [36]. More broadly, studies of attestation mechanisms for TEEs emphasize that the practical value of attestation depends on what is measured, how claims are encoded, and how a verifier evaluates them [38]. In this sense, attestation converts confidential execution from a local hardware property into a system property that can be consumed by remote tenants, orchestration systems, and key brokers.

2) *Broader Attestation Context in Arm Systems*: The same lesson appears in earlier TrustZone-oriented literature. Work on secure boot, trusted boot, mutual remote attestation, and runtime integrity measurement shows that deployable trust depends on evidence spanning both launch time and runtime [39]–[41]. For edge and IoT platforms, **PSA Certified** provides a broader framework for hardware-rooted assurance and attestation-oriented system design [42]. Earlier attestation systems such as **SWATT**, **SMART**, and **SEDA** reinforce the same point from a wider systems perspective: evidence must be attributable to a concrete execution context and fresh enough to support a real trust decision [43]–[45]. Within Arm CCA, attestation is therefore not an optional add-on; it is the mechanism that makes Realm isolation externally verifiable.

#### F. Comparative Perspective

1) *TrustZone versus Arm CCA*: The transition from **TrustZone** to **Arm CCA** is best understood as a change in

both threat model and enforcement granularity. TrustZone is effective when a secure-world software stack is itself part of the trusted boundary, but Arm CCA is designed for settings in which a powerful host stack must remain outside the confidentiality boundary [6]–[10], [15]. As a result, protection moves from a mostly binary world partition to an ownership model backed by GPT/GPC checks and Realm lifecycle management. The comparison is not that one mechanism replaces the other in all settings; rather, they target different deployment assumptions.

TABLE VII  
ARCHITECTURAL COMPARISON: TRUSTZONE AND ARM CCA

| Property                 | TrustZone                                          | Arm CCA                                                                         |
|--------------------------|----------------------------------------------------|---------------------------------------------------------------------------------|
| Primary Isolation Model  | Secure / Non-secure world split                    | Realm-based confidential execution with Root/Secure/Realm/Non-secure separation |
| Protection Granularity   | Platform-defined secure-world partitioning         | Granule ownership tracked through GPT/GPC                                       |
| Trusted Software Context | Secure-world firmware and services                 | RMM-mediated Realm lifecycle with host outside the data boundary                |
| Memory Enforcement Style | World-aware interconnect and secure memory regions | Hardware-checked ownership transitions for protected granules                   |
| Device Model             | Typically static secure-world device handling      | Realm-aware device and I/O mediation                                            |
| Attestation Practice     | Often platform- or vendor-specific                 | Structured evidence for Realm launch and platform state                         |

2) *Arm CCA in Comparative Context: AMD SEV-SNP* addresses a similar high-level problem by protecting guest workloads from a host with substantial management authority [46], [47]. The useful comparison is therefore not whether both systems support confidential workloads, but how they represent and enforce the protection boundary. In the Arm case, the cited architecture documents and systems papers emphasize explicit security states, Realm lifecycle management, and hardware ownership checks over physical memory [8]–[10]. By contrast, the comparison with **RISC-V PMP** highlights a different design axis: CCA uses table-backed ownership metadata, whereas PMP uses machine-mode configuration registers to define protected physical regions [9], [10], [48]. The latter remains an important primitive for smaller systems, but the former is better aligned with large, dynamic workloads that repeatedly change memory ownership and depend on coordinated device mediation.

#### G. Design Tradeoffs and Open Challenges

1) *Scalability, Complexity, and System Composition:* The literature surveyed above suggests that Arm CCA’s strengths also imply new engineering burdens. Table-backed ownership tracking and explicit lifecycle transitions are more expressive than coarse partitioning, but they require careful coordination among the architecture, the monitor, the host software stack, and devices [8]–[10], [15]. The emergence of systems such as **NanoZone**, **RContainer**, and **virtCCA** indicates that researchers are already exploring how to scale these mechanisms

to user space, containers, and virtualized deployments without losing the clarity of the protection boundary [21]–[23]. This suggests that scalability is not only a performance question; it is also a question of whether the management model remains understandable as deployments become more complex.

2) *Data-in-Use versus Data on the System Fabric:* A second recurring issue is that confidential execution cannot be reduced to CPU-local isolation. In Arm CCA specifically, this concern appears through the need to align Realm isolation with device mediation, interconnect protection, and attestation rather than through one standalone mechanism [9], [10], [12], [28], [29]. CXL and PCIe IDE are cited here as data-path and link-protection context, not as proof that Arm CCA alone secures every fabric endpoint. The architecture therefore addresses confidential computing as a composition problem: protected execution, memory ownership, I/O mediation, and trust evidence have to remain consistent across the entire platform.

#### H. Implications for Network Devices

The mechanisms surveyed in this section are directly relevant to modern network hardware. Network appliances increasingly combine general-purpose cores, **SmartNICs**, **DPUs**, storage engines, and accelerators on the same platform, and many of these components are **DMA-capable**. In such systems, protecting a confidential workload is not just a matter of isolating CPU execution; it also requires consistent mediation of memory ownership, device assignment, interrupts, and attestation evidence. Arm CCA is therefore particularly relevant to network equipment that performs virtualization, confidential packet processing, or multi-tenant offload, because the same platform must let the operator manage resources without automatically exposing tenant memory or accelerator state. Current accelerator and DPU work shows four complementary routes: protect the accelerator through a CCA-aware path, as in ACAI and CAGE; protect SoC devices through CCA-managed access regions, as in PORTAL; protect rack- or cloud-scale heterogeneous devices through a security controller, as in HETEE and CloudScale; or rely on device-local roots such as DPU OP-TEE/TPM building blocks, NIC-local roots, and accelerator-local TEEs [13], [25], [26], [49]–[53]. The DPU OP-TEE/TPM material is vendor evidence for building blocks, not proof of a complete production confidential-networking TEE.

#### I. Summary

The Arm literature surveyed here shows a clear progression from TrustZone-based TEEs to confidential-computing architectures with stronger assumptions about privileged adversaries. TrustZone established a system-wide isolation model based on Secure and Non-secure worlds, but its protection boundary is relatively coarse and often tied to a growing secure-world TCB. Arm CCA extends the design by introducing Realms, GPT/GPC-backed memory ownership, RMM-managed lifecycle control, device mediation through the I/O path, and structured attestation evidence [6]–[10], [15]. A

cross-cutting observation is that confidential computing in Arm is not defined by any single primitive. Its security properties emerge from the composition of memory ownership tracking, device mediation, and externally verifiable attestation, all of which must hold simultaneously if the workload is to remain protected from a powerful host environment.

## V. RISC-V CONFIDENTIAL COMPUTING

RISC-V confidential computing is best understood as the convergence of two historically separate lines of work. The first line is the open enclave literature, where researchers use RISC-V’s extensible ISA, privilege model, and physical-memory protection mechanisms to build small trusted monitors and enclave runtimes. The second line is the standardization-oriented confidential VM effort represented by CoVE and AP-TEE, where the target is no longer only an application enclave but a tenant VM protected from an untrusted host environment [3], [54], [55]. Boubakri et al. are used here as survey/background substrate; mechanism and status claims in this section rely on the original CoVE paper, AP-TEE draft, or other primary sources. This distinction is important for classification: PMP-based enclaves, tagged-memory enclaves, and CoVE-style TVMs share a motivation, but they do not expose the same lifecycle, device, or attestation contract.

### A. From Enclave Research to Confidential VMs

1) *Early RISC-V Enclave Design:* **Sanctum** is one of the earliest and most influential examples of using RISC-V to explore strong software isolation through minimal hardware extensions [56]. Its importance is not just that it provides an enclave system, but that it frames open hardware as a way to make TEE assumptions explicit. Instead of treating a proprietary platform as a fixed black box, Sanctum exposes the processor, cache, memory, and monitor assumptions that must hold for enclave isolation.

**Keystone** then provides a more reusable open-source RISC-V TEE framework built around a security monitor, PMP-based memory isolation, and runtime abstractions for enclaves [57]. Keystone is central to the survey because it shows the strengths and limitations of using standard RISC-V mechanisms for protected execution. PMP gives a simple and efficient physical-memory filter, but a small number of PMP regions and monitor-managed state are not the same as a cloud-scale confidential-VM memory ownership model. This is the point at which RISC-V enclave research begins to diverge from later CoVE/AP-TEE work.

2) *Scaling and Specializing the Enclave Boundary:* Subsequent RISC-V enclave systems explore different ways to scale, specialize, or strengthen the basic monitor-plus-memory-protection model. **TIMBER-V** uses tag-isolated memory to support fine-grained enclaves, which is especially relevant for embedded and compartmentalized systems where the protected unit may be smaller than a full VM [58]. **CURE** introduces customizable resilient enclaves, emphasizing configurable isolation and resilience tradeoffs [59]. **MI6** studies secure en-

claves in the context of a speculative out-of-order processor, making clear that the microarchitecture matters even when the ISA-level enclave abstraction appears simple [60].

**Penglai** addresses a different bottleneck: scalable memory protection for large numbers of enclaves, motivated by cloud and serverless deployment patterns [61]. **SPEAR-V** proposes low-overhead enclave primitives for RISC-V and is useful as a later point in the lineage because it studies practical primitive design rather than only monitor structure [62]. **Cerberus** focuses on formal and efficient enclave memory sharing, an issue that becomes increasingly important once protected domains need deliberate shared-memory communication rather than complete isolation [63]. **ACE** extends the discussion toward confidential computing for embedded RISC-V systems [64].

Across these works, the recurring lesson is that a TEE is not only an execution mode. It is a contract over memory ownership, entry and exit, metadata integrity, sharing, interrupt behavior, and attestation. Earlier enclave systems often solve a subset of that contract for a specific threat model. CoVE/AP-TEE attempts to standardize a larger contract for confidential VMs.

### B. CoVE and AP-TEE

1) *Threat Model and Architectural Role:* RISC-V CoVE targets a host-resilient confidential-VM model. The tenant workload executes in a trusted virtual machine while a host stack remains responsible for resource management, scheduling, and device orchestration [54]. This is conceptually close to Arm CCA in that both designs separate management authority from data visibility, but the RISC-V path is expressed through a different standardization structure and a more modular ecosystem. The AP-TEE specification defines the relevant ABI and lifecycle concepts, but it remains a draft and should be treated as a standardization work in progress rather than a ratified architecture [55].

The core AP-TEE objects are the **Trusted Security Monitor (TSM)**, the **TSM driver**, **Supervisor Domains**, and the **Trusted Virtual Machine (TVM)** [55]. The host retains the ability to allocate resources, but the TSM mediates TVM creation, memory assignment, entry, exit, and security-sensitive transitions. This moves RISC-V beyond the simple observation that PMP can isolate memory. A confidential VM needs a defined lifecycle, a way to donate and reclaim pages, an attestation path, and an interface that lets untrusted management software perform useful work without reading protected state.

2) *Memory Lifecycle and TVM State:* Memory lifecycle is the central mechanism that differentiates CoVE/AP-TEE from earlier PMP-only enclave explanations. The draft AP-TEE model includes memory **donation**, **reclaim**, and **sharing** semantics, which are needed because a TVM is dynamic: pages are created, mapped, unmapped, shared for communication, and eventually returned to the host [55]. These transitions are security-critical. If a page can move between host and TVM ownership without explicit state checks, stale mappings,



unclean metadata, or ambiguous sharing state can break the confidentiality boundary.

This is also where RISC-V confidential computing should not be reduced to memory encryption. The AP-TEE draft primarily specifies ownership, lifecycle, and access-control semantics; it does not by itself replace a memory-encryption and replay-protection taxonomy [65]. A complete platform may combine access control, encryption, integrity protection, and attestation, but those mechanisms answer different questions. Access control asks which agent may reach a physical page. Encryption asks what an observer of raw memory can learn. Integrity and replay protection ask whether stale or modified data can be detected. A precise survey has to keep those layers separate.

3) *Attestation and Evidence*: AP-TEE also requires an attestation story because tenants need external evidence before provisioning secrets to a TVM [55]. The evidence has to bind the platform, TSM, TVM initial state, and relevant configuration to a verifier-visible claim. This mirrors the broader attestation literature: a TEE claim is useful only if the measured state is well-defined, fresh, tied to the execution context that will receive secrets, and appraised by a verifier under a clear RATS-style role model [16], [37], [38].

In RISC-V systems, this evidence chain is especially important because the ecosystem encourages modular composition. A TVM may depend on a TSM, a platform root of trust, an IOMMU, interrupt controllers, device-security protocols, and firmware components from different vendors or projects. Attestation is the mechanism that turns that composition into something a remote tenant can evaluate.

### C. RISC-V Platform Primitives for Confidential VMs

RISC-V confidential computing depends on several architectural primitives that are security-relevant but not all confidential-computing mechanisms by themselves. Table VIII separates substrate specifications from the draft confidential-VM and trusted-I/O layers. This distinction prevents PMP, IOMMU, AIA, or IOPMP from being described as complete CoVE/AP-TEE equivalents merely because they are necessary parts of a platform.

The table also fixes the standardization hierarchy used in the remainder of this section. The IOMMU and AIA are treated as official architectural substrate, while IOPMP, AP-TEE, and CoVE-IO retain their draft or pre-ratification status. As a result, the survey can discuss how these pieces compose without implying that every primitive has the same maturity or claim strength.

### D. RISC-V Confidential I/O

1) *Why CoVE Needs I/O Standardization*: A confidential VM that cannot use real devices is not sufficient for cloud or edge deployment. CoVE-IO and TEE-I/O therefore extend the TVM discussion into device identity, DMA, MMIO, interrupts, and link protection [66]. The draft CoVE-IO specification is not ratified, but it is already useful as a map of the required

mechanisms. It introduces concepts such as trusted device interfaces and device/security managers while relying on lower-level building blocks including SPDMA, TDISP, PCIe IDE, IOMMU support, and trusted interrupt delivery [29], [66], [67], [70]. The TDISP citation is public PCI-SIG metadata for a member-gated ECN, so it supports only the device-interface lifecycle role unless a stronger public source is cited for a specific normative rule.

The design implication is straightforward: CoVE cannot stop at CPU-side page ownership. A device assigned to a TVM must be identified, measured or certified, placed into an appropriate trust state, constrained in the memory it can access, and connected to the TVM through an interrupt path that does not silently return control to the untrusted host. If any of those steps is missing, the I/O path becomes a privileged channel around the confidential-computing boundary.

2) *RISC-V IOMMU, IOPMP, AIA, and sIOPMP*: The RISC-V IOMMU and AIA specifications provide two of the required supporting layers. The IOMMU constrains device-side address translation and access permissions, while AIA provides the interrupt and MSI architecture that CoVE-IO can build on for trusted delivery [68], [69]. These specifications are not confidential-computing systems by themselves. Their role is to provide the architectural substrate on which a confidential I/O protocol can express and enforce device ownership.

IOPMP and sIOPMP address a related but distinct problem: scalable filtering of I/O transactions so that requester identity and memory-domain policy can be checked outside the CPU's ordinary memory-access path [71], [72]. For RISC-V confidential computing, this matters because DMA is the obvious way to bypass a CPU-centered protection story. A TVM memory page protected from host loads and stores is still exposed if a bus master can reach it without going through a domain-aware check.

### E. Comparison with Arm CCA

Table X shows why the comparison should be made at the confidential-VM layer rather than between CCA and bare PMP. PMP remains a useful primitive, especially for monitors and embedded TEEs, but CoVE/AP-TEE is the closer architectural analogue to Arm CCA. Both systems try to let an untrusted host manage resources without giving it data visibility. The difference is that Arm presents a more vertically integrated Realm architecture, whereas RISC-V standardization exposes a set of modular pieces that platform builders must assemble into a coherent protection boundary.

### F. Summary

RISC-V confidential computing has a rich pre-standardization lineage in open-source enclave systems and a rapidly developing standardization path in CoVE/AP-TEE. Earlier systems such as Sanctum, Keystone, TIMBER-V, CURE, MI6, Penglai, SPEAR-V, Cerberus, and ACE establish the design vocabulary for isolation, monitor structure, metadata, and memory sharing. CoVE/AP-TEE raises the abstraction to confidential VMs, with TVM lifecycle, TSM

TABLE VIII  
RISC-V PLATFORM PRIMITIVES FOR CONFIDENTIAL VMs

| Primitive       | Role                                                                                         | Confidential-computing body? | Maturity                                                       | Survey use                             |
|-----------------|----------------------------------------------------------------------------------------------|------------------------------|----------------------------------------------------------------|----------------------------------------|
| PMP/ePMP/Smepmp | CPU-side physical access control and monitor-enforced protection regions                     | No                           | Privileged architecture / extension substrate                  | Enclave and monitor baseline.          |
| H-extension     | Virtualization substrate for guest execution and host/guest separation                       | No                           | Architectural substrate in the privileged specification family | TVM execution substrate.               |
| AIA/IMSIC       | Interrupt and MSI architecture for virtualized interrupt delivery                            | No                           | AIA v1.0 ratified June 2023; revised 20250312 clarifications   | Trusted-MSI context for CoVE-IO.       |
| IOMMU           | Device-side address translation and DMA permission enforcement                               | No                           | RISC-V IOMMU v1.0.1 / 20260222 ratified-library release        | DMA isolation substrate.               |
| IOPMP           | Requester-aware transaction filtering outside the CPU path                                   | No                           | IOPMP v0.8.2 draft, not ratified                               | DMA bypass defense direction.          |
| AP-TEE/CoVE     | TVM, TSM, memory lifecycle, and ABI direction for confidential VMs                           | Yes                          | Draft/not ratified: AP-TEE v0.7 / CoVE proposal                | Core RISC-V confidential-VM direction. |
| CoVE-IO         | Trusted I/O ABI, device identity, TDI lifecycle, trusted MMIO/DMA, and trusted MSI direction | Yes                          | Draft/not ratified: CoVE-IO v0.3.0                             | Trusted-I/O direction for TVMs.        |

TABLE IX  
RISC-V CoVE/AP-TEE LIFECYCLE AND STATE-MACHINE SUMMARY

| Lifecycle phase                    | State object or transition                                                                                  | Evidence status                                                                                               | Survey claim boundary                                                                                          |
|------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| TVM creation and admission         | TSM admits a TVM, establishes launch state, and separates host resource management from TVM data visibility | E3: AP-TEE v0.7 draft / CoVE proposal direction [54], [55]                                                    | Use as RISC-V confidential-VM direction, not as ratified ABI behavior.                                         |
| Memory donation and acceptance     | Host-managed pages become TVM-owned pages through TSM-mediated lifecycle transitions                        | E3: AP-TEE draft lifecycle vocabulary [55]                                                                    | Supports draft state-machine comparison with Arm CCA; detailed ABI and corner cases remain draft.              |
| Sharing, reclaim, and teardown     | Pages move between private, shared, reclaimed, or destroyed states under TSM policy                         | E3 draft plus RISC-V lineage papers for background sharing models [55], [63]                                  | Useful for identifying lifecycle requirements; not yet a ratified cross-platform contract.                     |
| Attestation and evidence appraisal | TVM launch/runtime evidence is encoded and appraised by verifier/relying-party roles                        | E0: RATS/EAT generic roles and token format; E3: AP-TEE platform-specific evidence direction [16], [37], [55] | Can claim standard attestation vocabulary and draft RISC-V direction, not complete production verifier policy. |
| Device-interface admission         | TDI/TDM/DSM-style actors bind a device interface to a TVM and compose SPDM, TDISP, IDE, and IOMMU controls  | E3: CoVE-IO v0.3.0; E0 SPDM/IOMMU/AIA substrate; E5 TDISP public metadata [66]–[70]                           | Strongest claim is required-layer mapping; the CoVE-IO interface state machine is still draft.                 |
| Trusted MSI and interrupt routing  | Interrupt ownership follows device assignment and TVM ownership rather than host-only routing               | E0 AIA substrate with E3 CoVE-IO trusted-I/O direction [66], [69]                                             | Use as trusted-interrupt design direction; do not claim ratified trusted-MSI semantics.                        |

TABLE X  
ARM CCA AND RISC-V CoVE/AP-TEE DESIGN MAPPING

| Design Point                            | Arm CCA                                             | RISC-V CoVE/AP-TEE                                            |
|-----------------------------------------|-----------------------------------------------------|---------------------------------------------------------------|
| Protected workload Management component | Realm RMM                                           | Trusted Virtual Machine TSM and TSM driver                    |
| Host role                               | Resource management outside the Realm data boundary | Resource management outside the TVM data boundary             |
| Memory ownership                        | GPT/GPC and Realm lifecycle                         | TVM memory donation, reclaim, sharing, and TSM-mediated state |
| Device path                             | SMMU and realm-aware device mediation               | CoVE-IO, IOMMU, IOPMP, AIA, SPDM, TDISP, PCIe IDE             |
| Standard status                         | Arm architecture/specification path                 | AP-TEE and CoVE-IO drafts, not ratified                       |

mediation, memory ownership transitions, and attestation as first-class concerns. CoVE-IO then extends the same reasoning to devices and networks. The resulting lesson is that RISC-V confidential computing is not one mechanism but a platform composition problem: CPU privilege, memory

ownership, I/O translation, interrupt routing, device identity, and attestation all have to agree on the same security boundary.

## VI. CONFIDENTIAL I/O AND NETWORK DATA-PATH DEFENSE

Confidential computing protects data in use, but real workloads rarely remain inside CPU registers and local DRAM. They use NICs, DPUs, SmartNICs, storage controllers, GPUs, NPUs, CXL-attached memory, RDMA paths, and virtual switches. These components move data through DMA engines, device queues, interrupts, MMIO windows, link layers, and sometimes device-local runtimes. A CPU TEE or confidential VM therefore provides only a partial boundary unless the I/O path is integrated into the same trust model [4], [12], [49], [50], [52], [66], [74].

This section treats confidential I/O and network defense as a separate survey lane. It does not cover generic network security. Instead, it focuses on mechanisms that let a confidential workload safely use a device, network path, remote memory tier, or accelerator without handing plaintext or control authority back to an untrusted host.

TABLE XI  
ARM CCA VERSUS RISC-V CoVE/AP-TEE CLAIM-STRENGTH COMPARISON

| Claim axis                             | Arm CCA evidence                                                                                                                    | RISC-V CoVE/AP-TEE evidence                                                                                                      | Allowed survey claim                                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Confidential-VM abstraction            | E1 OSDI design/verification plus E0 Arm architecture and RMM specifications [8], [9], [15]                                          | E3 CoVE/AP-TEE proposal and draft material [54], [55]                                                                            | Compare architectural goals and roles; do not imply equal standard maturity.                                             |
| Memory ownership lifecycle             | E0/E1 support for Realm granule ownership, GPT/GPC checks, and RMM-mediated transitions [8]–[10]                                    | E3 draft support for TVM memory donation, sharing, reclaim, and TSM mediation [55]                                               | Arm supports stronger mechanism/state claims; RISC-V supports draft direction and gap analysis.                          |
| Attestation evidence                   | E0 Arm CCA/RMM plus E0 RATS/EAT vocabulary for role and token semantics [9], [15], [16], [37]                                       | E0 RATS/EAT vocabulary plus E3 AP-TEE platform evidence direction [16], [37], [55]                                               | Compare evidence-chain requirements; platform-specific RISC-V claims remain draft.                                       |
| Device and trusted-I/O path            | E0 SMMU substrate and E1/E3 Arm CCA device/accelerator research such as ACAI, PORTAL, CAGE, and Devlore [12]–[14], [25], [26]       | E0 IOMMU/AIA/SPDM substrate, E3 CoVE-IO, and E5 TDISP metadata [66]–[70]                                                         | Claim both need device identity, DMA control, interrupts, and lifecycle; RISC-V trusted I/O remains a draft composition. |
| Implementation and deployment evidence | E1 systems and public Arm/Linux documentation show research prototypes and architecture-facing implementation paths [8], [11], [73] | E1 lineage systems plus E3 AP-TEE/CoVE-IO drafts show design direction rather than ratified deployment contracts [55]–[57], [66] | Use RISC-V lineage as background and draft standardization evidence, not as proof of completed production parity.        |

### A. Threat Model and Data-Path Surfaces

1) *Why Device Paths Break CPU-Only Isolation:* The basic device threat is simple: DMA-capable devices can access memory without executing CPU load and store instructions. If the platform protects a confidential VM only through CPU page tables, PMP, GPT/GPC, or monitor-managed state, a device with insufficiently constrained DMA authority can still reach protected pages [12], [68], [71]. The same concern applies to MMIO registers and queue metadata. A host that cannot read a TVM’s memory directly may still observe descriptors, packet buffers, completion queues, or device status if those structures are not assigned and protected as part of the confidential boundary.

Interrupts create a second surface. Device completion and error events must be delivered to the protected workload without letting the host spoof, suppress, or misroute security-critical signals. Work such as Devlore makes this explicit for confidential VMs: interrupt ownership and delivery are part of device ownership, not a secondary OS scheduling detail [14]. For RISC-V, AIA and CoVE-IO provide the architectural vocabulary for trusted MSI and interrupt handling, while Arm platforms rely on SMMU and interrupt controller integration in the broader CCA system model [12], [66], [69]. Attack-only work such as HECKLER shows why this boundary matters: malicious interrupt delivery can undermine confidential VMs, but this survey keeps HECKLER in the excluded attack bibliography and uses it only as motivation for interrupt ownership requirements.<sup>1</sup>

2) *Network and Offload-Specific Risks:* Network devices add their own complications. A confidential workload may use a virtual NIC, SR-IOV virtual function, SmartNIC, DPU, programmable switch path, or RDMA queue pair. In each case, the sensitive state is not only application payload. It includes descriptors, keys, queue ownership, completion events, flow steering rules, packet headers, and device firmware state. A secure offload design must therefore answer four questions:

which device or function is the endpoint, what code and configuration is running there, which memory can it access, and how packets or DMA buffers are protected while crossing the fabric.

FOLIO illustrates one possible tradeoff for confidential VM networking by aiming for high-performance networks without requiring trusted I/O devices [75]. This is useful because it separates two design paths. One path tries to make the device part of the trusted computing base through assignment, measurement, and secure transport. The other path tries to avoid trusting the device by keeping plaintext and sensitive control inside the confidential VM while still obtaining high throughput. Both directions belong in this survey because they address the same confidential-computing bottleneck: network performance should not require silently expanding the trusted boundary to an opaque device stack.

Network endpoint identity is a separate question from device identity. Even if a NIC or DPU can be identified through SPDM-like mechanisms, a tenant still needs to know whether the application channel carrying secrets terminates inside the intended TEE software. TLS+RA addresses this endpoint problem by binding remote attestation evidence to TLS channel establishment, so that a secure channel is not merely adjacent to a TEE quote but cryptographically tied to the attested endpoint [76]. This matters for confidential vNIC, proxy, gateway, and DPU-hosted services because TLS termination can otherwise become a place where plaintext exits the confidential boundary.

SmartNIC and NIC-local trust work pushes the same question lower in the stack. **S-NIC** shows that a multi-tenant SmartNIC needs hardware isolation for on-NIC RAM, cache, accelerators, DMA, and bus bandwidth before offloaded network functions can be treated as tenant-confidential components [74]. **TNIC** takes a different route by placing a minimal root of trust at the NIC, providing transferable authentication and non-equivocation for trustworthy distributed systems [52]. **OSMOSIS** adds a background resource-management view: multi-tenant SmartNIC kernels contend for on-NIC com-

<sup>1</sup><https://www.usenix.org/conference/usenixsecurity24/presentation/schluter>; metadata is kept in `survey/excluded_attack_reference.bib`.

pute, DMA, egress bandwidth, and local memory, so a programmable offload endpoint also needs scheduling and QoS mechanisms before it can be operationally usable in a multi-tenant setting [77]. These works are not substitutes for VM memory protection, SPDM, or TDISP. They clarify what a NIC or DPU must provide if it becomes a real endpoint of a confidential data path.

#### *B. Protocol Layer: Identity, Session Security, and Device State*

##### *1) SPDM as Device Identity and Measurement Substrate:*

The **Security Protocol and Data Model (SPDM; DMTF DSP0274 v1.4.0, selected as this survey's SPDM snapshot)** provides a protocol for device authentication, measurement retrieval, version and capability negotiation, and session establishment [67], [78]. In confidential-computing systems, SPDM is valuable because the host's view of a device is not enough. A tenant or trusted manager needs evidence about the device identity and state before allowing that device to participate in a protected data path.

Secured Messages Using SPDM (DMTF DSP0277 v2.0.0) adds confidentiality and integrity for protocol exchanges after session establishment [79]. SPDM over TCP (DMTF DSP0287 v1.0.0) extends the transport setting so that SPDM can be used beyond only local bus-oriented interactions [80]. These specifications do not themselves define a complete confidential I/O architecture. Their role is to supply the identity, measurement, and secure-channel substrate that higher level mechanisms such as TDISP, CoVE-IO, and trusted I/O designs can rely on.

*2) TDISP and Trusted Device Interfaces:* **TDISP** addresses a more specific lifecycle problem: how a PCIe device interface can be placed into a TEE-aware trust state and assigned to a confidential workload [70]. The cited PCI-SIG material is a public ECN/metadata page whose download target is member-gated, so this survey uses it only for TDISP's public existence, affected-specification metadata, and device-interface lifecycle role rather than for unreleased normative details such as exact messages, state-machine rules, or conformance requirements. The later public **TDISP eXtended TEE (XT)** ECN metadata is treated under the same E5 boundary: it supports only the public fact that PCI-SIG lists IDE, ATS, and TDISP as affected PCIe 6.x areas, not gated XT/TE normative rules [81]. Device identity alone is not enough. The platform also needs a way to bind a particular device interface, function, or trusted device interface to the workload that will use it. TDISP therefore belongs at the device-interface lifecycle layer, whereas SPDM belongs at the identity and secure-session layer.

The RISC-V CoVE-IO draft uses this kind of structure explicitly. It refers to trusted device interfaces, device managers, security managers, SPDM, TDISP, PCIe IDE, IOMMU controls, and trusted MSI as cooperating parts of a confidential I/O design [66]. The important survey point is that no single layer is sufficient. A measured device without constrained DMA can still leak memory. A constrained DMA path without device identity can still attach the wrong endpoint. A protected

link without lifecycle control can still carry traffic for an interface that was not securely assigned.

*3) Attested Service Channels:* SPDM and TDISP focus on the device and device-interface layers, but confidential workloads also expose application-level endpoints. A key broker, model-serving endpoint, storage proxy, packet-processing service, or DPU-hosted TLS termination service may all need to prove that the network channel reaches the expected protected code. **TLS+RA** integrates remote attestation into the TLS handshake while keeping certificate-based endpoint identity and attestation-based platform evidence independently deployable and independently failing [76]. This gives the survey a seventh layer: attested service-channel binding. It is distinct from device identity because the question is not only "which device is present" but "where does this application connection terminate."

**aDNS** addresses the neighboring discovery and naming problem: how a client learns that a domain name is bound to a confidential service whose attestation satisfies that domain's policy. Transparent Attested DNS requires a TEE-backed service to present hardware attestation before registering keys and obtaining certificates for names in the relevant zone, and records policies, attestations, and certificates in a public append-only log [82]. This complements TLS+RA rather than replacing it. TLS+RA binds a channel to an attested endpoint; aDNS binds service names, certificate issuance, and verifier policy to attestation evidence. Neither mechanism constrains DMA, assigns a device interface, or proves the application-level semantics of the service.

#### *C. Data Plane: DMA, Links, and Fabrics*

*1) IOMMU, SMMU, IOPMP, and Address Authority:* DMA mediation is the first enforcement layer for confidential I/O. Arm systems use SMMU-based translation and protection to constrain device accesses in the system memory path [12]. RISC-V systems use the IOMMU specification for device address translation and IOPMP-style mechanisms for requester-aware transaction filtering [68], [71]. Academic work such as sIOPMP emphasizes the scalability pressure: protected systems may have many devices, many domains, and frequent mapping changes, so the I/O protection mechanism must be both precise and efficient [72].

These mechanisms should be classified as access-control and translation mechanisms. They do not automatically encrypt data on a link, validate the device's internal firmware, or prove the endpoint state to a remote verifier. Their job is to ensure that a bus master can access only the memory and MMIO resources assigned to it. This classification prevents a common survey error: equating any I/O protection with a complete trusted I/O system.

*2) PCIe IDE, CXL, RDMA, and Remote Memory Paths:* Link and fabric protection answer a different question: what happens to data as it traverses PCIe, CXL, RDMA, or related interconnects. PCIe IDE provides integrity and data encryption for PCIe traffic, which is link protection rather than a complete trusted-I/O stack [29]. CXL changes the memory hierarchy by

allowing memory expansion, pooling, and tiering beyond local DRAM [28], [31], [32]. RDMA and remote paging systems move application memory over the network or fabric with minimal CPU involvement [33]. These CXL/RDMA/fabric sources are background substrate and data-path evidence, not proof that a confidential-computing boundary remains intact by themselves; they define paths that a confidential-computing defense must cover.

The design consequence is that confidential workloads need a data-path view, not only a CPU view. A page that is protected while resident in local memory may later move through a CXL fabric, be staged through an RDMA-capable NIC, or be referenced by a device queue. Each transition introduces a new enforcement point. The platform must decide whether protection comes from link encryption, DMA access control, endpoint TEE support, software copying through trusted buffers, or a combination of these mechanisms.

Secure storage over network fabrics adds freshness and crash-consistency questions to this path. **Hazel** studies NVMe-over-Fabrics storage under a confidential-computing threat model and combines counter leasing, NVMe metadata, a disaggregated Merkle-tree design, and BlueField-3 offload to provide confidentiality, integrity, and freshness without changing the NVMe-oF protocol [83]. This kind of work is useful for the survey because it shows that a protected I/O path must sometimes defend not only in-flight packets, but also remote persistent state and replay freshness.

#### *D. Trusted I/O Systems and Device TEEs*

1) *SEV-TIO and VM-Oriented Trusted I/O*: AMD SEV-TIO provides a concrete industry design point for trusted I/O in encrypted-VM systems [84]. It is useful in this survey even though the main architectural focus is Arm and RISC-V, because it clarifies the general pattern: a confidential VM needs device assignment, device trust establishment, protected DMA, interrupt handling, and attestation-compatible state. These requirements reappear in CoVE-IO and in Arm CCA-oriented device work.

The SEV-TIO comparison also helps avoid an overly narrow interpretation of trusted I/O. Trusted I/O is not merely "attach a device to a VM." It is the combination of device identity, interface lifecycle, memory access restriction, link security, interrupt delivery, and evidence that a verifier can evaluate. Different platforms divide those responsibilities differently, but the required security properties are similar.

2) *Accelerator TEEs and Confidential Offload*: Accelerator TEEs push the same problem into device-local computation. The accelerator may execute kernels, hold model weights, stage tensors, manage queues, or perform packet-processing offload. A CPU confidential VM cannot automatically protect that state once it leaves the CPU package. The accelerator itself must either become part of the trusted boundary or be treated as untrusted with protocols that keep secrets inside the confidential workload.

ACAI studies accelerator protection in the Arm CCA context [13]. PORTAL studies protected Arm CCA device access

for mobile SoCs, using CCA memory isolation to avoid repeated shared-buffer encryption [26]. CAGE adapts GPU and FPGA accelerator workflows to Arm CCA with shadow tasks and GPC/GPT-backed checks [25]. StrongBox is an earlier Arm GPU TEE baseline that uses TrustZone and Stage-2 translation to protect unified-memory GPU computation [27]. ITX demonstrates confidential computing within an AI accelerator, giving a concrete example of device-local confidential execution [53]. The broader accelerator TEE lineage shows several different protection routes: Graviton adds GPU secure contexts and command-processor ownership checks, Telekine makes cloud-GPU streams independent of secret-dependent behavior, Honeycomb uses static validation to shrink GPU TEE hardware requirements, and ShEF moves the same question to cloud FPGAs with secure boot, attestation, and a configurable Shield [85]–[88]. XpuTEE is tracked as a recent CPU/GPU heterogeneous TEE baseline, but only with source-limited metadata and abstract-level evidence in this survey; SGX-FPGA is also kept as source-limited metadata and is not used for strong mechanism claims [89], [90]. The SoC-FPGA SoK is useful for classifying edge and SoC-FPGA TEE gaps, but its claims still need primary system papers for mechanism detail [91]. The accelerator TEE SoK organizes this space around device identity, memory and bus protection, runtime TCB, scheduling, and attestation [4]. These works are important for network defense as well because modern NICs, DPUs, and SmartNICs increasingly resemble accelerators: they execute firmware and programmable code, maintain queues, and transform data before the CPU sees it.

3) *SmartNIC, DPU, and Rack-Scale Offload*: SmartNICs and DPUs sit between the device-interface layer and the application service layer. They can expose SR-IOV functions and virtual switches to the host, run embedded Arm-side control software, terminate tunnels, process packets, or mediate storage traffic. HETEE and later cloud-scale work show one route for protecting accelerator and DPU-like offload: place a security controller on the data path so that non-TEE devices can be assigned, measured, and cleaned up as part of a confidential workload lifecycle [49], [50]. Commercial DPU documentation shows another route: DPU-local roots such as OP-TEE/fTPM and replay-protected storage can provide platform identity and key protection building blocks [51]. That vendor evidence should be used carefully, because the cited BlueField documentation describes fTPM support and its limitations rather than a complete production confidential networking TEE, and it does not by itself bind a workload channel, DMA policy, or TDISP/SPDM lifecycle to the DPU state.

S-NIC, TNIC, and OSMOSIS fill three additional gaps. S-NIC answers how a SmartNIC can isolate tenant network functions from other functions and from the NIC OS, TNIC answers how a NIC can provide a small, verifiable trust root for networked protocols, and OSMOSIS answers how a SmartNIC can multiplex compute, DMA, egress, and memory resources among tenants with explicit QoS policies [52], [74], [77]. Hazel then shows how DPU/SmartNIC offload can be

TABLE XII  
CONFIDENTIAL I/O LIFECYCLE AND STATE-MACHINE SUMMARY

| Phase                         | Representative state transition                                                                              | Primary evidence status                                                                                                        | Survey claim boundary                                                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint identity and session | Requester/responder capabilities, certificates, measurements, and secured-session setup                      | E0: DMTF DSP0274 v1.4.0 selected SPDm snapshot; secured-message and TCP bindings are protocol substrate [67], [79], [80]       | Supports device identity, measurement, negotiation, and secure sessions; does not by itself assign the device or constrain DMA.    |
| Interface admission           | Device interface is requested, configured, and bound to a protected workload or device manager               | E3: CoVE-IO v0.3.0 draft; E5: TDISP/TDISP XT public metadata only [66], [70], [81]                                             | Trusted-device-interface lifecycle is a draft/metadata-backed direction here; no gated TDISP normative state rule is claimed.      |
| DMA and MMIO authority        | I/O page tables, SMMU/IOMMU mappings, or IOPMP-style filters are installed for the assigned endpoint         | E0: Arm SMMU and RISC-V IOMMU substrate; E3: IOPMP draft; E1: sIOPMP system paper [12], [68], [71], [72]                       | Strong access-control/translation claim for the cited primitive, not a complete endpoint-trust claim.                              |
| Link and fabric protection    | PCIe/CXL/RDMA/NVMe-oF path is encrypted, integrity protected, or routed through a protected data-path design | Spec and background substrate for PCIe IDE/CXL plus E1/E3 systems such as FO-LIO, ODRP, and Hazel [28], [29], [33], [75], [83] | Supports data-path/fabric risk analysis; must be composed with endpoint identity and memory authority before becoming trusted I/O. |
| Interrupt and event delivery  | Completion/error/MSI events are delivered to the intended protected domain without host confusion            | E0: AIA/IMSIc interrupt substrate; E3/E1 research direction for trusted MSI and device-interrupt protection [14], [66], [69]   | Claim is currently strongest as a required lifecycle layer; complete production semantics remain platform-specific.                |
| Operation and re-attestation  | Endpoint state, workload channel, or service channel is rechecked while secrets are in use                   | E0 RATS/EAT roles and token format; E1 TLS+RA/channel-binding evidence; E4 vendor DPU roots [16], [37], [51], [76]             | Evidence must be bound to the workload channel; vendor building blocks are not complete confidential-networking TEEs.              |
| Teardown and cleanup          | Interface is stopped, DMA authority revoked, device state reset or zeroized, and evidence invalidated        | E1 accelerator/offload systems and E3 CoVE-IO direction [49], [50], [66]                                                       | Cleanup is necessary for lifecycle safety but remains an open cross-device validation gap.                                         |

used in a secure storage data path while preserving freshness properties [83]. Together these papers make the DPU/NIC category more precise: one must distinguish local resource isolation, endpoint attestation, protocol-level non-equivocation, resource-management QoS, and storage freshness.

For survey classification, these systems are neither ordinary firewall/IDS papers nor complete replacements for CoVE-IO or TDISP. They are evidence that network offload devices are becoming programmable trust endpoints. A secure confidential networking stack must therefore compose DPU-local roots, device-interface lifecycle, DMA restrictions, link protection, and attested application-channel termination.

#### E. Taxonomy

Table XIII captures the classification used in the rest of this survey. The table is intentionally layered because confidential I/O failures are often composition failures. A system can have SPDm but weak DMA protection, PCIe IDE but no trusted device-interface lifecycle, or a device TEE but an ambiguous interrupt path. The practical question is whether all relevant layers preserve the same ownership boundary.

#### F. Summary

Confidential I/O and network defense is the point where confidential computing becomes a whole-platform problem. SPDm and secured messages provide identity, measurement, and session foundations. TDISP and CoVE-IO introduce trusted device-interface lifecycle concepts, with TDISP cited here only through public metadata rather than gated normative state-machine rules, and CoVE-IO still a draft. TLS+RA-style protocols bind application channels to attested endpoints, while aDNS adds attested naming, certificate issuance, and service-policy discovery. SMMU, IOMMU, IOPMP, and sIOPMP constrain DMA authority. AIA and trusted MSI address interrupt delivery. PCIe IDE, CXL, RDMA, NVMe-oF,

and remote-memory systems define the fabric paths that protected data may cross, but they are background substrate unless composed with device identity, lifecycle, and DMA controls. Accelerator, SmartNIC, NIC, and DPU work such as ACAI, PORTAL, CAGE, StrongBox, ITX, HETEE, CloudScale, S-NIC, TNIC, OSMOSIS, Hazel, and BlueField OP-TEE/FTPM shows that device-local, NIC-local, storage-path, or controller-mediated execution can contribute building blocks for a confidential boundary. A survey that focuses only on CPU TEE mechanisms would miss this entire class of defenses; for modern Arm and RISC-V confidential-computing platforms, these mechanisms are core rather than peripheral.

## VII. SECURITY DESIGN MECHANISMS IN ARM AND RISC-V

Hardware security mechanisms are increasingly being exposed as architectural primitives rather than as optional software conventions. Across modern Arm and RISC-V systems, security-relevant invariants such as indirect-control validation, page-table authority, physical-memory ownership, and privileged-access constraints are moving into the hardware-software contract itself [48], [92]. For a survey of hardware security, this shift matters because it changes where defenses live: instead of relying only on compiler instrumentation or operating-system policy, the architecture now provides direct mechanisms for constraining control flow, limiting memory authority, and mediating access across privilege and device boundaries. The following sections survey these mechanisms by design objective, focusing on runtime integrity, privilege and memory isolation, memory-safety hardening, and the system-integration issues that determine whether a mechanism remains effective once devices, debug paths, and accelerators are considered.

TABLE XIII  
CONFIDENTIAL I/O AND NETWORK DEFENSE TAXONOMY

| Layer                                 | Primary Question                                                                                          | Representative Mechanisms                                                                                                                                                                                                          | Survey Role                                                                                                                                                                     |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Device identity and measurement       | Which endpoint is participating, and what state can it prove?                                             | SPDM, SPDm secured messages, SPDm over TCP                                                                                                                                                                                         | Establish evidence and secure sessions for device trust.                                                                                                                        |
| Device-interface lifecycle            | Which interface is assigned to the confidential workload?                                                 | TDISP, trusted device interface state, CoVE-IO TDI/TDM/DSM                                                                                                                                                                         | Bind a device function or interface to a protected domain.                                                                                                                      |
| Attested service channel              | Does the application channel terminate in the attested protected endpoint?                                | TLS+RA, aDNS, RA-TLS-style endpoint binding, verifier policy                                                                                                                                                                       | Bind service names, certificates, KBS endpoints, proxies, and DPU-hosted TLS termination to TEE evidence.                                                                       |
| DMA and MMIO authority                | Which memory and registers may the device access?                                                         | SMMU, RISC-V IOMMU, IOPMP, sIOPMP                                                                                                                                                                                                  | Enforce address translation and transaction filtering.                                                                                                                          |
| Interrupt and event delivery          | Can completion and error events be delivered without host spoofing or confusion?                          | AIA, trusted MSI, Devlore-style interrupt ownership                                                                                                                                                                                | Keep device ownership coherent after asynchronous events.                                                                                                                       |
| Link and fabric protection            | What protects traffic across PCIe, CXL, RDMA, and remote-memory paths?                                    | PCIe IDE, CXL fabric controls, protected RDMA/remote paging designs                                                                                                                                                                | Protect data once it leaves CPU-local memory paths.                                                                                                                             |
| NIC-local root and function isolation | Can a SmartNIC/DPU isolate tenant functions, multiplex resources, or provide a trusted network primitive? | S-NIC, TNIC, OSMOSIS resource-management substrate, vendor DPU OP-TEE/TPM building blocks                                                                                                                                          | Treat NICs and DPUs as explicit but incomplete trust endpoints that still need attestation, lifecycle, DMA, channel binding, and resource policy.                               |
| Confidential storage data path        | Can remote storage preserve confidentiality, integrity, and freshness?                                    | Hazel, protected NVMe-oF metadata, DPU crypto offload                                                                                                                                                                              | Extend confidential I/O beyond packets into persistent remote state.                                                                                                            |
| Device-local confidential execution   | Does the accelerator or offload engine hold secrets or execute trusted code?                              | ACAI, PORTAL, CAGE, StrongBox, Graviton, Telekine, Honeycomb, ShEF, ITX, XpuTEE source-limited metadata, SGX-FPGA source-limited metadata, HETEE, CloudScale, vendor BlueField OP-TEE/TPM building blocks, accelerator TEE designs | Extend or avoid extending the TCB to accelerators and offload devices; do not treat vendor building blocks or source-limited metadata as complete confidential-networking TEEs. |

#### A. Control-Flow and Memory-Safety Mechanisms

1) *Arm Runtime-Integrity Mechanisms*: Arm’s recent A-profile architecture incorporates several mechanisms that aim to make common exploitation primitives harder to realize in practice [92]. **Branch Target Identification (BTI)** constrains indirect branches to architecturally valid landing pads, reducing the usable gadget space for jump-oriented and related control-flow attacks. **Pointer Authentication (PAC)** protects pointers with cryptographic authentication codes, making pointer substitution and reuse attacks more difficult. **Guarded Control Stack (GCS)** adds hardware support for protecting return-address state. These mechanisms address different parts of the control-flow problem, but together they represent a clear design direction: indirect control transfer is no longer treated as something that software may validate only if it chooses to.

Arm applies a similar idea to memory-safety failures. **Memory Tagging Extension (MTE)** associates tags with memory granules and checks tag consistency on pointer dereference, allowing the hardware to detect classes of spatial and temporal memory-safety violations with relatively low overhead [92]. From a survey perspective, the important point is not that any one mechanism eliminates memory corruption, but that the architecture increasingly offers built-in support for narrowing the gap between a latent software bug and a reliable control-hijacking exploit.

Recent systems evaluation shows that these primitives should still be treated as partial isolation building blocks rather than complete security boundaries. Chen et al. compare Intel

MPK, Arm MTE, Arm PAC, and Arm Morello for fine-grained subsystem isolation, and find that practical low-overhead isolation also depends on software transparency, efficient zero-copy exchange, cross-core synchronization of access rights, enough isolation domains, and first-class revocation support [93]. This result is useful for the survey’s hardware-design lane because it turns MTE, PAC, and capability-style mechanisms from a feature checklist into an engineering question: can the mechanism preserve an isolation boundary under frequent sharing, revocation, and cross-core execution?

2) *RISC-V Control-Flow and Protection Primitives*: The RISC-V architectural material surveyed here shows a related, though more modular, direction. The privileged and associated architectural specifications document control-flow integrity capabilities through extensions such as **Zicfilp**, which constrains indirect-control targets through landing-pad semantics, and **Zicfiss**, which provides hardware support for protecting return-address state [48]. CVA6-CFI provides implementation and evaluation evidence for those RISC-V CFI extensions in one processor context, but it is not evidence of a complete TEE, memory-confidentiality, DMA-isolation, or CoVE/AP-TEE boundary [94]. Rather than embedding one vertically integrated protection model, the RISC-V ecosystem exposes a set of composable primitives that can be incorporated into different classes of systems.

This modularity is particularly visible in the treatment of memory authority. In the cited RISC-V specifications, the most established protection story centers on privilege separation and physical-memory access control, especially through **Physical**

**Memory Protection (PMP)** and its extensions, rather than on a single Arm-MTE-style mainstream memory-tagging mechanism [48]. The result is a design space in which control-flow hardening and memory isolation exist, but are often assembled as part of a larger platform composition.

### *B. Privilege, Physical Memory, and Domain Isolation*

#### *1) Arm Privilege Hierarchy and System-Wide Isolation:*

Arm’s protection model combines conventional privilege separation with system-wide domain isolation. The exception-level hierarchy from EL0 to EL3 separates application, kernel, hypervisor, and monitor software, while **TrustZone** extends this structure with Secure and Non-secure worlds enforced across the SoC [6], [7], [92]. Additional mechanisms such as **PAN** and **PXN** constrain privileged software inside a single world by preventing inappropriate access to or execution from lower-privilege mappings [92]. These mechanisms are important because many real attacks do not begin by crossing a world boundary; they begin by exploiting excessive authority inside an already privileged context.

At the system level, Arm’s more recent confidential-computing extensions push isolation further into the memory system. **Arm CCA** and **RME** use hardware checks over physical-memory ownership to protect **Realms** even in the presence of a powerful host stack [8]–[10]. In other words, the architecture is moving from privilege-only isolation toward coordinated CPU, memory, and device mediation. This progression is part of a broader trend in secure hardware design: once DMA-capable devices and privileged software are part of the threat model, core-local access checks are not enough.

#### *2) RISC-V Privilege Modes and PMP-Based Isolation:*

RISC-V organizes protection around privilege modes and a register-programmed physical-memory filter. **PMP** allows machine mode to define access permissions for physical-memory regions, thereby constraining S-mode and U-mode software [48]. Extensions such as **ePMP** and **Smepmp** strengthen this model by refining the handling of execute permissions and reducing the amount of implicitly trusted machine-mode behavior [48]. The design is comparatively simple and well suited to embedded systems, monitors, and smaller TEEs where a limited number of protected regions is acceptable.

The main limitation of this style appears when system complexity increases. Large SoCs, confidential-computing platforms, and accelerator-rich systems must often coordinate CPU isolation with **DMA** control and device-visible memory ownership. Recent work such as **sIOPMP** therefore studies how scalable I/O protection can complement CPU-side isolation for TEEs [72]. This is an important design point for survey work: in RISC-V platforms, the strength of the final security story depends heavily on how PMP-like CPU mechanisms are composed with peripheral protection.

### *C. Page Tables, Capabilities, and Memory-Protection Semantics*

*1) PTE and Page-Table Permissions:* Page tables are a first-order hardware defense mechanism, not only a virtual-

memory convenience. In Arm systems, page-table attributes and privileged-access controls define whether a mapping is readable, writable, executable, accessible from lower privilege, or usable as an instruction source [92]. Mechanisms such as **PXN**, **UXN**, and **PAN** are therefore best classified as page-table and privilege-hardening defenses. They reduce the ambient authority of privileged software by making kernel execution from user-controlled mappings and privileged access to user memory explicit policy choices rather than default behaviors.

RISC-V has an analogous but differently organized permission story. RISC-V page-table entries encode read, write, execute, user, global, accessed, and dirty state, while **PMP/ePMP/Smepmp** provide physical-address constraints below or beside the virtual-memory layer [48]. The important survey distinction is that page-table permissions, PMP-style filters, and confidential-computing ownership metadata are different layers. Page tables express address-translation and per-page software protection policy. PMP and IOPMP constrain physical-memory and transaction authority. CCA GPT/GPC and CoVE/AP-TEE lifecycle state express confidential-domain ownership. Confusing these layers leads to incorrect claims about what a mechanism actually defends.

*2) Capability and Tagging Architectures:* Capability and tagging architectures take a more semantic approach to memory authority. **CHERI**IoT applies capability-based memory safety to embedded devices and demonstrates how bounds, permissions, and object provenance can be represented in hardware-enforced capabilities [95]. **RV-CURE** explores a RISC-V capability architecture for full memory safety [96]. These mechanisms are relevant to confidential-computing platforms because many attacks on protected workloads still begin as ordinary memory-safety failures inside the trusted or protected domain. A Realm or TVM can hide data from the host while still being vulnerable to corruption inside its own software stack.

The correct classification is therefore defense-in-depth. Capability and tagging mechanisms do not replace CCA, CoVE, TrustZone, or device assignment. Instead, they reduce the probability that code inside a protected domain can be exploited to misuse the domain’s legitimate authority. This is a recurring theme in modern hardware security: isolation defines who is outside the boundary, while memory-safety and capability mechanisms reduce abuse by code that is already inside it.

#### *3) Memory Encryption, Integrity, and Replay Protection:*

Memory encryption should also be separated from access-control mechanisms. A memory-encryption design addresses what an observer of raw memory or a memory bus can learn, while integrity and replay-protection designs address whether modified or stale data can be detected [65]. AMD SEV-SNP is an important modern example because it combines encrypted-VM deployment with stronger protection against host-controlled memory-management attacks [47]. Arm CCA and RISC-V CoVE/AP-TEE, by contrast, primarily specify ownership, lifecycle, and access-control semantics in the cited



architectural materials [9], [55]. A complete product may combine these layers, but a survey should not treat them as interchangeable.

Recent architecture work also shows that these layers can be deliberately co-designed rather than simply stacked. **Voodoo** combines memory tagging, authenticated encryption, and DRAM error correction through a MAGIC-style construction, using tag-dependent authentication to protect encrypted memory while also preserving correction capabilities [97]. Its role in this survey is narrow but important: it is evidence that memory-safety metadata, confidentiality, integrity, and reliability can share a hardware primitive. It is not evidence for Realm or TVM lifecycle state, attestation, DMA authorization, or trusted device assignment.

This taxonomy is especially important for Arm and RISC-V confidential-computing comparison. GPT/GPC, PMP, IOPMP, and IOMMU checks decide whether an access is allowed. Link protection such as PCIe IDE protects traffic on a fabric. Memory encryption protects data representation outside the processor or protected boundary. Integrity and anti-replay mechanisms protect freshness and tamper detection. These mechanisms compose, but none of them alone proves that a workload is confidential against the full host, device, and fabric threat model.

#### *D. Trusted Execution, Boot, and Trust Anchors*

1) *TEE-Oriented Design Patterns*: Trusted-execution support remains one of the clearest ways in which hardware security mechanisms appear as complete platform designs rather than isolated instructions. In the Arm ecosystem, **TrustZone** provides the classical two-world TEE model and has been used as the basis for a large body of systems work and security analysis [6], [7], [17], [18]. Arm CCA extends this line further by introducing **Realms** for confidential workloads under stronger host-side adversary assumptions [8]–[10]. More broadly, the comparison between TrustZone-style TEEs and CCA-style confidential execution shows how trusted-execution design has moved from isolating a privileged security service toward protecting workloads against stronger software adversaries.

Verified enclave monitors are an important intermediate point in that lineage. **Komodo** shows how enclave-management policy can be moved out of fixed hardware and into a machine-checked software monitor while relying on smaller hardware primitives for isolation, memory protection, and attestation support [98]. Its role here is not to stand in for Arm CCA or CoVE/AP-TEE, but to show why verification and small monitor interfaces matter for TEE design.

2) *Boot-Time Trust and Measured Launch*: Trusted execution is closely connected to how a platform establishes trust at boot time. Work on secure boot, trusted boot, and remote attestation for Arm TrustZone-based devices demonstrates that the security of the runtime environment depends on a measured launch chain rather than on isolation alone [39]–[41]. At a broader systems level, **SWATT**, **SMART**, and **SEDA** show recurring design requirements for attestation and

roots of trust: measurements must be attributable, fresh, and tied to a concrete execution context [43]–[45]. For hardware designers, this means that the relevant trust anchor is not simply a privileged mode or a secure memory region; it is the combination of launch-time measurement, protected execution, and verifiable system state.

Runtime attestation adds another layer: the verifier may need evidence about the path a program actually took, not only the code image that booted. **C-FLAT** introduces control-flow attestation with TrustZone-protected measurement, **LO-FAT** moves that measurement into hardware to avoid software instrumentation overhead, and **VRASED** demonstrates a formally verified hardware/software RA co-design for simple embedded devices [99]–[101]. These papers do not replace RATS/EAT or platform-specific CCA/CoVE evidence profiles; they explain why runtime evidence and root-of-trust implementation details matter.

#### *E. I/O, DMA, and Accelerator Mediation*

1) *Arm System Mediation Beyond the CPU*: Once devices and accelerators are part of the threat model, hardware security depends on more than CPU privilege checks. Arm’s system architecture therefore includes I/O mediation through the **SMMU**, which is especially important when confidential workloads or secure services interact with **DMA**-capable peripherals [12]. The same point is visible in Arm CCA-oriented systems work, where accelerator protection and device-interrupt delivery are treated as part of the core isolation problem rather than as optional external policy [13], [14]. This is a crucial design lesson for secure hardware surveys: a platform that protects the CPU but leaves devices outside the security model has only partial isolation.

2) *RISC-V I/O Protection as a Complementary Layer*: RISC-V reaches the same requirement through a more modular route. The base privileged architecture centers on CPU-side protection, while later work such as **SIOPMP** studies scalable I/O protection as a complementary mechanism for TEEs and protected domains [48], [72]. This separation reinforces the broader pattern seen throughout the chapter: RISC-V often exposes a toolkit whose final security properties depend on disciplined composition, whereas Arm more often presents these pieces as parts of a vertically integrated platform-security story.

#### *F. Attestation and Security Lifecycle Mechanisms*

1) *Attestation as a Hardware-Supported Design Mechanism*: Attestation is not only a protocol feature; it is also a hardware-design concern because the architecture must expose stable roots of trust, measurable state, and protection domains whose claims can be verified. The **RATS** architecture standardizes attestation roles and conceptual messages, and the **EAT** standard provides a general structure for carrying attestation claims; neither replaces platform-specific evidence semantics [16], [37]. Broader work on attestation mechanisms for TEEs emphasizes that the usefulness of those claims depends on what the hardware and trusted runtime are able to

measure and report [38]. In this sense, attestation belongs in a survey of hardware security design because it links platform mechanisms to external trust decisions.

2) *Lifecycle Security Frameworks*: The same design logic appears in platform frameworks such as **PSA Certified**, which treats hardware-rooted security, attestation, and lifecycle management as part of one assurance story rather than as isolated controls [42]. This kind of framework is important because it extends the discussion beyond the presence of security mechanisms to their use across provisioning, update, and operational phases. For surveys of secure hardware design, lifecycle treatment matters because many platforms fail not through the absence of a mechanism, but through inconsistent handling of that mechanism across deployment stages.

#### G. Debug, Trace, and System Integration

1) *Security-Relevant Auxiliary Components*: Architectural security mechanisms are not limited to branches, page tables, and privilege bits. Arm’s documentation on trace makes clear that debug and observability features such as **ETM** provide detailed execution visibility that is valuable for validation and diagnostics, but that same visibility must be tightly controlled in security-sensitive deployments [102]. This illustrates a broader systems lesson: every auxiliary subsystem that can observe execution or preserve state across contexts can become part of the attack surface if it is not incorporated into the protection model.

2) *Composition as the Central Design Problem*: Across both Arm and RISC-V, the mechanisms surveyed above point to the same conclusion. Secure hardware design is not achieved by enabling a single extension in isolation. Control-flow hardening, memory-access constraints, privilege separation, and device mediation have to compose correctly across the whole platform [9], [10], [48], [72], [92]. This is also why architecture manuals and systems papers increasingly discuss memory ownership, I/O paths, and monitor logic in the same breath: the boundary that matters in practice is the one that survives contact with the full machine, not just the CPU pipeline.

#### H. Cross-Architecture Comparison

Table XIV summarizes the main mechanism categories surveyed in this section. The goal is not to claim that the two ecosystems are identical, but to show that they increasingly address comparable security objectives through different integration styles.

The comparison suggests a consistent difference in emphasis. The Arm material surveyed here presents a more vertically integrated security stack in which runtime integrity, world/domain isolation, and system-wide mediation are documented as parts of one platform story [9], [10], [92]. The RISC-V material, by contrast, emphasizes composable building blocks centered on privilege modes and PMP-based physical-memory control, with additional system protections layered on through complementary mechanisms and proposals [48], [72]. This is not a value judgment so much as a design

TABLE XIV  
CROSS-ARCHITECTURE SECURITY MECHANISM MAPPING

| Mechanism Category                   | Arm                                                          | RISC-V                                                                            |
|--------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Forward-edge control-flow protection | BTI                                                          | Zicfilp; CVA6-CFI prototype evidence                                              |
| Return-address protection            | GCS / PAC-assisted hardening                                 | Zicfiss; CVA6-CFI prototype evidence                                              |
| Memory bug detection                 | MTE                                                          | Protection centered on privilege and memory-access control in the cited materials |
| Primary structure                    | privilege EL0–EL3 plus Secure/Non-secure system partitioning | M/S/U privilege modes                                                             |
| Physical-memory isolation            | TrustZone system partitioning; CCA/RME ownership checks      | PMP, ePMP, Smepmp                                                                 |
| Page-table hardening                 | PAN, PXN, UXN, privileged translation controls               | PTE R/W/X/U/A/D permissions and privilege-aware translation                       |
| Capability or tagging memory safety  | MTE and capability-adjacent ecosystem work                   | CHERIOT, RV-CURE, tag/capability research systems                                 |
| System-level I/O protection          | SMMU-mediated system integration in Arm platforms            | IOMMU, IOPMP, sIOPMP, AIA-based interrupt support                                 |

distinction: one ecosystem foregrounds integration, while the other foregrounds modular composition.

#### I. Implications for Secure Hardware Design

For hardware designers, the practical lesson is that secure systems must be evaluated as whole-machine constructions. A platform with strong control-flow protections can still fail if debug visibility is unmanaged. A platform with strict CPU isolation can still fail if a device bypasses memory protections through **DMA**. A platform with clean privilege layering can still fail if the architecture leaves too much ambient authority in the most privileged software context. The mechanisms surveyed in this section therefore support the same conclusion reached elsewhere in this paper: modern hardware security depends on coordinated enforcement across the instruction set, the memory system, and the device path.

#### J. Summary

The architectures surveyed here show a broad movement toward hardware-enforced security invariants. In Arm, this is visible in the combination of BTI, PAC, GCS, MTE, page-table hardening, privilege constraints, TrustZone, and CCA/RME-based system isolation [6], [9], [10], [92]. In RISC-V, the same trend appears through control-flow integrity extensions, PMP-family physical-memory protection, page-table permissions, capability/tagging research, and complementary proposals for scalable I/O mediation [48], [72], [95], [96]. A cross-cutting observation is that the core design problem is no longer whether hardware offers security features at all, but whether

those features compose into a coherent platform-level protection model once privileged software, accelerators, and device traffic are part of the system.

## REFERENCES

- [1] M. Schneider, R. J. Masti, S. Shinde, S. Capkun, and R. Perez, “Sok: Hardware-supported trusted execution environments,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.12742>
- [2] M. Li, Y. Yang, G. Chen, M. Yan, and Y. Zhang, “Sok: Understanding design choices and pitfalls of trusted execution environments,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1600–1616. [Online]. Available: <https://doi.org/10.1145/3634737.3644993>
- [3] M. Boubakri and B. Zouari, “A survey of risc-v secure enclaves and trusted execution environments,” *Electronics*, vol. 14, no. 21, p. 4171, 2025. [Online]. Available: <https://www.mdpi.com/2079-9292/14/21/4171>
- [4] C. Wang and J. Huang, “Sok: Analysis of accelerator tee designs,” in *Network and Distributed System Security Symposium*, 2026, nDSS 2026 paper; use when discussing accelerator TEEs rather than general CCA only. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2026-f1424-paper.pdf>
- [5] J. Shi, J. Gu, Y. Xia, and H. Chen, “Serverless functions made confidential and efficient with split containers,” in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 1091–1110. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/shi-jiacheng>
- [6] ARM Limited, “Arm security technology: Building a secure system using trustzone technology,” ARM Ltd., Tech. Rep., 2009. [Online]. Available: <https://documentation-service.arm.com/static/5f212796500e883ab8e74531>
- [7] S. Pinto and N. Santos, “Demystifying arm trustzone: A comprehensive survey,” *ACM Comput. Surv.*, vol. 51, no. 6, Jan. 2019. [Online]. Available: <https://doi.org/10.1145/3291047>
- [8] X. Li, X. Li, C. Dall, R. Gu, J. Nieh, Y. Sait, and G. Stockwell, “Design and verification of the arm confidential compute architecture,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 465–484. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/li>
- [9] ARM Limited, “Arm confidential compute architecture specification,” ARM Ltd., Tech. Rep., 2025. [Online]. Available: <https://developer.arm.com/documentation/den0125/latest>
- [10] —, “Realm management extension architecture specification,” ARM Ltd., Tech. Rep., 2021. [Online]. Available: <https://documentation-service.arm.com/static/60d32e78677cf7536a55bad7>
- [11] Linux Kernel Community, “Arm confidential compute architecture documentation,” <https://docs.kernel.org/arch/arm64/arm-cca.html>, 2023.
- [12] ARM Limited, “Arm system memory management unit architecture specification (smmuv3),” ARM Ltd., Tech. Rep., 2022. [Online]. Available: <https://developer.arm.com/documentation/den0070/latest>
- [13] S. Sridhara, A. Bertschi, B. Schlüter, M. Kuhne, F. Aliberti, and S. Shinde, “Acai: Protecting accelerator execution with arm confidential computing architecture,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.15986>
- [14] A. Bertschi, S. Sridhara, M. Kuhne, B. Schlüter, F. Groschupp, C. Thorens, N. Dutty, S. Capkun, and S. Shinde, “Devlore: Device interrupt protection for confidential vms,” 2026. [Online]. Available: <https://arxiv.org/abs/2408.05835>
- [15] ARM Limited, “Realm management monitor specification,” ARM Ltd., Tech. Rep., 2025. [Online]. Available: <https://developer.arm.com/documentation/den0137/latest/>
- [16] L. Lundblade, G. Mandyam, J. O’Donoghue, and C. Wallace, “The Entity Attestation Token (EAT),” RFC 9711, Apr. 2025. [Online]. Available: <https://www.rfc-editor.org/info/rfc9711>
- [17] L. Guan, P. Liu, X. Xing, X. Ge, S. Zhang, M. Yu, and T. Jaeger, “Trustshadow: Secure execution of unmodified applications with arm trustzone,” in *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services*, 2017, pp. 488–501. [Online]. Available: <https://doi.org/10.1145/3081333.3081349>
- [18] D. Cerdeira, N. Santos, P. Fonseca, and S. Pinto, “Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1416–1432.
- [19] T. Wu, S. Xiong, E. Manino, G. Stockwell, and L. C. Cordeiro, “Verifying components of arm(r) confidential computing architecture with esbmc,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.04375>
- [20] Y. Zhang, Y. Hu, Z. Ning, F. Zhang, X. Luo, H. Huang, S. Yan, and Z. He, “SHELTER: Extending arm CCA with isolation in user space,” in *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2023, pp. 6257–6274. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/zhang-yiming>
- [21] Q. Zhou, W. Cao, X. Jia, P. Liu, S. Zhang, J. Chen, S. Xu, and Z. Song, “Rcontainer: A secure container architecture through extending arm cca hardware primitives,” in *32nd Annual Network and Distributed System Security Symposium (NDSS 2025)*. The Internet Society, 2025. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/rcontainer-a-secure-container-architecture-through-extending-arm-cca-hardware-primitives/>
- [22] X. Xu, W. Wang, Y. Wu, C. Wang, H. Zhu, H. Ma, Z. Min, Z. Pang, R. Hou, and Y. Jin, “virtcca: Virtualized arm confidential compute architecture with trustzone,” 2026. [Online]. Available: <https://arxiv.org/abs/2306.11011>
- [23] S. Liu, Y. Gao, M. Zhang, and J. Wang, “Nanozone: Scalable, efficient, and secure memory protection for arm cca,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.07034>
- [24] S. Liu, Z. Jiang, J. Wang, W. Zhou, K. Sun, Z. Chen, and Y. Xie, “More granular, less trust: Enforcing intra-process isolation with arm cca in an untrusted management environment,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 12 507–12 522, 2025.
- [25] C. Wang, K. Lu, F. Zhang, Y. Deng, K. Leach, J. Cao, Z. Ning, S. Yan, T. Wei, and Z. He, “Building confidential accelerator computing environment for Arm CCA,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 1308–1325, 2026. [Online]. Available: <https://ira.lib.polyu.edu.hk/handle/10397/117541>
- [26] F. Sang, J. Lee, X. Zhang, and T. Kim, “PORTAL: Fast and secure device access with Arm CCA for modern Arm mobile system-on-chips (SoCs),” in *Proceedings of the 46th IEEE Symposium on Security and Privacy*, 2025, pp. 4099–4116. [Online]. Available: [https://sang.fan/assets/papers/portal\\_sp25.pdf](https://sang.fan/assets/papers/portal_sp25.pdf)
- [27] Y. Deng, C. Wang, S. Yu, S. Liu, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang, “Strongbox: A GPU TEE on Arm endpoints,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 769–783. [Online]. Available: <https://fengweiz.github.io/paper/strongbox-ccs22.pdf>
- [28] Compute Express Link Consortium, “Compute express link specification,” CXL Consortium, Tech. Rep., 2023. [Online]. Available: <https://computeexpresslink.org/wp-content/uploads/2024/02/CXL-3.1-Specification.pdf>
- [29] PCI-SIG, “Pci express integrity and data encryption (ide),” PCI Special Interest Group, Tech. Rep., 2025. [Online]. Available: <https://pcisig.com/specification-overview/pci-express-base>
- [30] UEFI Forum, “Advanced configuration and power interface (acpi) specification,” UEFI Forum, Tech. Rep., 2021. [Online]. Available: [https://uefi.org/htmlspecs/ACPI\\_Spec\\_6\\_4.html/](https://uefi.org/htmlspecs/ACPI_Spec_6_4.html/)
- [31] D. Gouk, S. Lee, M. Kwon, and M. Jung, “Direct access, high-performance memory disaggregation with directcxl,” in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, 2022, pp. 287–294. [Online]. Available: <https://www.usenix.org/conference/atc22/presentation/gouk>
- [32] Y. Zhong, D. S. Berger, C. Waldspurger, R. Wee, I. Agarwal, R. Agarwal, F. Hady, K. Kumar, M. D. Hill, M. Chowdhury, and A. Cidon, “Managing memory tiers with CXL in virtualized environments,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 37–56. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yuhong>
- [33] Z. Wang, X. Wei, J. Gu, H. Xie, R. Chen, and H. Chen, “ODRP: on-demand remote paging with programmable RDMA,” in *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 2025. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/wang-zixuan>

- [34] M. Kuhne, S. Sridhara, A. Bertschi, N. Dutly, F. Aliberti, S. Capkun, and S. Shinde, "Bringing confidential computing to android," in *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys '26)*. New York, NY, USA: Association for Computing Machinery, Jun. 2026. [Online]. Available: <https://aster-cca.github.io/>
- [35] J. Chen, Z. Mi, Y. Xia, H. Guan, and H. Chen, "CPC: Flexible, secure, and efficient CVM maintenance with confidential procedure calls," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 1065–1082. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/chen-jiahao>
- [36] Trusted Computing Group, "Hardware requirements for a device identifier composition engine," Trusted Computing Group, Tech. Rep. Level 00 Revision 78, 2018, tCG Published specification. [Online]. Available: [https://trustedcomputinggroup.org/wp-content/uploads/Hardware-Requirements-for-Device-Identifier-Composition-Engine-r78\\_For-Publication.pdf](https://trustedcomputinggroup.org/wp-content/uploads/Hardware-Requirements-for-Device-Identifier-Composition-Engine-r78_For-Publication.pdf)
- [37] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," RFC 9334, Jan. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9334>
- [38] J. Ménétrey, C. Göttel, M. Pasin, P. Felber, and V. Schiavoni, "An exploratory study of attestation mechanisms for trusted execution environments," 2022. [Online]. Available: <https://arxiv.org/abs/2204.06790>
- [39] Z. Ling, H. Yan, X. Shao, J. Luo, Y. Xu, B. Pearson, and X. Fu, "Secure boot, trusted boot and remote attestation for arm trustzone-based iot nodes," *Journal of Systems Architecture*, vol. 119, p. 102240, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762121001661>
- [40] Y. Chen, Y. Ding, J. Zhang, and L. Song, "Mra-ima: Enhanced mutual remote attestation based on arm trustzone," in *2024 9th International Conference on Computer and Communication Systems (ICCCS)*, 2024, pp. 1278–1284.
- [41] J. Mao and X. Chang, "Pdrima: A policy-driven runtime integrity measurement and attestation approach for arm trustzone-based tee," 2025. [Online]. Available: <https://arxiv.org/abs/2512.06500>
- [42] ARM Limited, "Psa certified security framework," ARM Ltd., Tech. Rep., 2023. [Online]. Available: <https://developer.arm.com/documentation/den0128/latest>
- [43] A. Seshadri, A. Perrig, L. van Doorn, and P. Khosla, "SWATT: software-based attestation for embedded devices," in *IEEE Symposium on Security and Privacy, 2004. Proceedings. 2004*, 2004, pp. 272–282.
- [44] K. M. E. Defrawy, G. Tsudik, A. Francillon, and D. Perito, "Smart: Secure and minimal architecture for (establishing dynamic) root of trust," in *Network and Distributed System Security Symposium*, 2012. [Online]. Available: <https://api.semanticscholar.org/CorpusID:909934>
- [45] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, "Seda: Scalable embedded device attestation," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 964–975. [Online]. Available: <https://doi.org/10.1145/2810103.2813670>
- [46] Advanced Micro Devices, "Secure encrypted virtualization (sev)," AMD, Tech. Rep., 2017. [Online]. Available: <https://www.amd.com/en/developer/sev.html>
- [47] AMD, "Secure encrypted virtualization secure nested paging (sev-snp)," AMD Ltd., Tech. Rep., 2020. [Online]. Available: <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>
- [48] RISC-V Foundation, *The RISC-V Instruction Set Manual: Privileged Architecture*, 2025. [Online]. Available: <https://docs.riscv.org/reference/isa/attachments/riscv-privileged.pdf>
- [49] J. Zhu, R. Hou, X. Wang, W. Wang, J. Cao, B. Zhao, Z. Wang, Y. Zhang, J. Ying, L. Zhang, and D. Meng, "Enabling rack-scale confidential computing using heterogeneous trusted execution environment," in *2020 IEEE Symposium on Security and Privacy*, 2020, pp. 1450–1465. [Online]. Available: <https://ieeexplore.ieee.org/document/9152787>
- [50] A. Dhar, S. Sridhara, S. Shinde, S. Capkun, and R. Andri, "Confidential computing with heterogeneous devices at cloud-scale," in *2024 Annual Computer Security Applications Conference*, 2024, pp. 102–116. [Online]. Available: <https://www.acsac.org/2024/program/final/s297.html>
- [51] NVIDIA, "NVIDIA BlueField Operation: fTPM over OP-TEE," NVIDIA, Tech. Rep., 2025, blueField BSP documentation. [Online]. Available: <https://docs.nvidia.com/networking/display/bluefieldbsp480/ftpm%2Bover%2Bop-tee>
- [52] D. Giantsidi, J. Pritzi, F. Gust, A. Katsarakis, A. Koshiba, and P. Bhatotia, "TNIC: A trusted NIC architecture," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025, pp. 1282–1301. [Online]. Available: <https://dse.in.tum.de/wp-content/uploads/2025/02/TNIC-ASPLOS-2025.pdf>
- [53] K. Vaswani, S. Volos, C. Fournet, A. N. Diaz, K. Gordon, B. Vembu, S. Webster, D. Chisnall, S. Kulkarni, G. Cunningham, R. Osborne, and D. Wilkinson, "Confidential computing within an AI accelerator," in *2023 USENIX Annual Technical Conference*, 2023, pp. 615–630. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/vaswani>
- [54] R. Sahita, A. Patra, V. Shanbhogue, S. Ortiz, A. Bresticker, D. Reid, A. Khare, and R. Kanwal, "Cove: Towards confidential computing on risc-v platforms," 2023. [Online]. Available: <https://arxiv.org/abs/2304.06167>
- [55] RISC-V Non-ISA AP-TEE Contributors, "Risc-v application-processor trusted execution environment specification," RISC-V International, Tech. Rep., 2024, draft/not ratified: AP-TEE v0.7 / RC2 for ARC review; CoVE/AP-TEE specification. [Online]. Available: <https://github.com/riscv-non-isa/riscv-ap-tee/releases/tag/v0.7>
- [56] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal hardware extensions for strong software isolation," in *25th USENIX Security Symposium (USENIX Security 16)*. Austin, TX: USENIX Association, Aug. 2016, pp. 857–874. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/costan>
- [57] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, "Keystone: An open framework for architecting trusted execution environments," in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3342195.3387532>
- [58] S. Weiser, M. Werner, F. Brasser, M. Malenko, S. Mangard, and A.-R. Sadeghi, "TIMBER-V: Tag-isolated memory bringing fine-grained enclaves to RISC-V," in *Network and Distributed System Security Symposium*, 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/timber-v-tag-isolated-memory-bringing-fine-grained-enclaves-to-risc-v/>
- [59] R. Bahmani, F. Brasser, G. Dessouky, P. Jauernig, M. Klimmek, A.-R. Sadeghi, and E. Stapf, "Cure: A security architecture with customizable and resilient enclaves," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.15866>
- [60] T. Bourgeat, I. Lebedev, A. Wright, S. Zhang, Arvind, and S. Devadas, "Mi6: Secure enclaves in a speculative out-of-order processor," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 42–56. [Online]. Available: <https://arxiv.org/abs/1812.09822>
- [61] E. Feng, X. Lu, D. Du, B. Yang, X. Jiang, Y. Xia, B. Zang, and H. Chen, "Scalable memory protection in the penglai enclave," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 2021, pp. 275–294. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/feng>
- [62] D. Schrammel, M. Waser, L. Lamster, M. Unterguggenberger, and S. Mangard, "Spear-v: Secure and practical enclave architecture for risc-v," in *Proceedings of the 18th ACM Asia Conference on Computer and Communications Security*, 2023, pp. 1178–1192. [Online]. Available: <https://doi.org/10.1145/3579856.3590264>
- [63] D. Lee, K. Cheang, A. Thomas, C. Lu, P. Gaddamadugu, A. Vahldiek-Oberwagner, M. Vij, D. Song, S. A. Seshia, and K. Asanovic, "Cerberus: A formal approach to secure and efficient enclave memory sharing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022. [Online]. Available: <https://people.eecs.berkeley.edu/~sseshia/pubs/b2hd-leeccheang-ccs22.html>
- [64] W. Ozga, G. D. H. Hunt, M. V. Le, L. Gäher, A. Shinnar, E. R. Palmer, H. Jamjoom, and S. Dragone, "Ace: Confidential computing for embedded risc-v systems," 2025. [Online]. Available: <https://arxiv.org/abs/2505.12995>

- [65] M. Henson and S. Taylor, "Memory encryption: A survey of existing techniques," *ACM Computing Surveys*, vol. 46, no. 4, 2014. [Online]. Available: <https://doi.org/10.1145/2566673>
- [66] RISC-V Non-ISA AP-TEE-IO Contributors, "Risc-v coe-io specification," RISC-V International, Tech. Rep., 2026, draft/not ratified: CoVE-IO v0.3.0. [Online]. Available: <https://github.com/riscv-non-isa/riscv-ap-tee-io/releases/tag/v0.3.0>
- [67] DMTF SPDm Working Group, "Security Protocol and Data Model (SPDM) specification," DMTF, Tech. Rep. DSP0274, 2025, dMTF DSP0274 v1.4.0, selected as this survey's SPDm snapshot. [Online]. Available: <https://www.dmtf.org/dsp/DSP0274>
- [68] RISC-V Non-ISA IOMMU Contributors, "Risc-v iommu architecture specification," RISC-V International, Tech. Rep., 2026, rISC-V IOMMU Base Architecture v1.0 and standard extensions ratified; current RISC-V Ratified Specifications Library release is v1.0.1 / 20260222, February 2026. [Online]. Available: [https://docs.riscv.org/r/reference/iommu/\\_attachments/riscv-iommu.pdf](https://docs.riscv.org/r/reference/iommu/_attachments/riscv-iommu.pdf)
- [69] RISC-V International, "Risc-v advanced interrupt architecture specification," RISC-V International, Tech. Rep., 2023, aIA v1.0 ratified in June 2023; current ratified-library document includes 20250312 clarifications. [Online]. Available: [https://docs.riscv.org/reference/aia/\\_attachments/riscv-interrupts.pdf](https://docs.riscv.org/reference/aia/_attachments/riscv-interrupts.pdf)
- [70] PCI-SIG, "TEE device interface security protocol (TDISP)," PCI-SIG, Tech. Rep., 2022, pCI-SIG Engineering Change Notice metadata; public PDF unavailable/member-gated. [Online]. Available: <https://pcisig.com/PCI%20Express/ECN/Base/TEEDeviceInterfaceSecurityProtocol>
- [71] RISC-V IOPMP Task Group, "Risc-v iopmp architecture specification," RISC-V International, Tech. Rep., 2026, version 0.8.2 draft, not ratified. [Online]. Available: <https://github.com/riscv-non-isa/riscv-iopmp/releases/tag/v0.8.2>
- [72] E. Feng, D. Feng, D. Du, Y. Xia, W. Zheng, S. Zhao, and H. Chen, "siopmp: Scalable and efficient i/o protection for tees," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 1061–1076. [Online]. Available: <https://doi.org/10.1145/3620665.3640378>
- [73] A. Bertsch and S. Shinde, "Opencca: An open framework to enable arm cca research," 2025. [Online]. Available: <https://arxiv.org/abs/2506.05129>
- [74] Y. Zhou, M. Wilkening, J. Mickens, and M. Yu, "SmartNIC security isolation in the cloud with S-NIC," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 851–869. [Online]. Available: <https://minlanyu.seas.harvard.edu/writeup/eurosys24.pdf>
- [75] M. Li, S. Srivastava, and M. Yan, "Bridge the future: High-performance networks in confidential VMs without trusted I/O devices," 2024. [Online]. Available: <https://arxiv.org/abs/2403.03360>
- [76] C. Weinhold, M. U. Sardar, I. Mihalcea, Y. Deshpande, H. Tschofenig, Y. Sheffer, T. Fossati, and M. Roitzsch, "Separate but together: Integrating remote attestation into TLS," in *2025 USENIX Annual Technical Conference*. USENIX Association, 2025, pp. 1319–1326. [Online]. Available: <https://www.usenix.org/conference/atc25/presentation/weinhold>
- [77] M. Khalilov, M. Chrapek, S. Shen, A. Vezzu, T. Benz, S. D. Girolamo, T. Schneider, D. D. Sensi, L. Benini, and T. Hoefler, "OSMOSIS: Enabling Multi-Tenancy in datacenter SmartNICs," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 247–263. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/khalilov>
- [78] DMTF SPDm Working Group, "Security Protocol and Data Model (SPDM) architecture white paper," DMTF, Tech. Rep. DSP2058, 2024, version 1.3.0. [Online]. Available: <https://www.dmtf.org/standards/spdm>
- [79] —, "Secured messages using SPDm specification," DMTF, Tech. Rep. DSP0277, 2025, dMTF Standard, Version 2.0.0. [Online]. Available: <https://www.dmtf.org/dsp/DSP0277>
- [80] —, "SPDM over TCP binding specification," DMTF, Tech. Rep. DSP0287, 2024, dMTF Standard, Version 1.0.0. [Online]. Available: <https://www.dmtf.org/dsp/DSP0287>
- [81] PCI-SIG, "TDISP extended TEE (XT) extensions," PCI-SIG, Tech. Rep., 2025, e5 public metadata only: PCIe 6.x ECN released 2025-04-22; member-gated download; public page states affected IDE, ATS, and TDISP areas. [Online]. Available: <https://pcisig.com/tdisp-extended-tee-xt-extensions>
- [82] A. Delignat-Lavaud, C. Fournet, K. Vaswani, M. Costa, S. Clebsch, and C. M. Wintersteiger, "Transparent attested DNS for confidential computing services," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 1129–1148. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/delignat-lavaud>
- [83] M. Chrapek, M. Orenbach, A. Atamli, M. Copik, M. Khalilov, F. Alder, and T. Hoefler, "Hazel: Secure and efficient disaggregated storage," 2026. [Online]. Available: <https://arxiv.org/abs/2510.18756>
- [84] AMD, "AMD SEV-TIO: Trusted I/O for secure encrypted virtualization," AMD, Tech. Rep., 2023. [Online]. Available: <https://docs.amd.com/v/u/en-US/sev-tio-whitepaper>
- [85] S. Volos, K. Vaswani, and R. Bruno, "Graviton: Trusted execution environments on GPUs," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 2018, pp. 681–696. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/volos>
- [86] T. Hunt, Z. Jia, V. Miller, A. Szekely, Y. Hu, C. J. Rossbach, and E. Witchel, "Telekine: Secure computing with cloud GPUs," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020, pp. 817–833. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/hunt>
- [87] H. Mai, J. Zhao, H. Zheng, Y. Zhao, Z. Liu, M. Gao, C. Wang, H. Cui, X. Feng, and C. Kozyrakis, "Honeycomb: Secure and efficient GPU executions via static validation," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, 2023, pp. 155–172. [Online]. Available: <https://www.usenix.org/conference/osdi23/presentation/mai>
- [88] M. Zhao, M. Gao, and C. Kozyrakis, "ShEF: Shielded enclaves for cloud FPGAs," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 1070–1085. [Online]. Available: <https://arxiv.org/abs/2103.03500>
- [89] S. Fan, Z. Hua, Y. Xia, and H. Chen, "XpuTEE: A high-performance and practical heterogeneous trusted execution environment for GPUs," *ACM Transactions on Computer Systems*, vol. 43, no. 1–2, pp. 1–27, 2025, e5 source-limited in this survey; local PDF unavailable; DOI metadata and abstract verified. [Online]. Available: <https://dl.acm.org/doi/10.1145/3719653>
- [90] K. Xia, Y. Luo, X. Xu, and S. Wei, "SGX-FPGA: Trusted execution environment for CPU-FPGA heterogeneous architecture," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 301–306, e5 source-limited metadata record; public PDF unavailable. [Online]. Available: <https://www.researchwithrutgers.com/en/publications/sgx-fpga-trusted-execution-environment-for-cpu-fpga-heterogeneous>
- [91] G. Perkins, B. Macht, L. L. Ritzdorf, T. R. Crane, B. J. LaMeres, C. Izurieta, and A. M. Reinhold, "SoK: Trusted execution in SoC-FPGAs," in *6th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA 2024)*, 2024, peer-reviewed conference paper; arXiv v1 posted 2025-03-20. [Online]. Available: <https://arxiv.org/abs/2503.16612>
- [92] ARM Limited, *ARM Architecture Reference Manual, for A-profile architecture*, ARM Ltd., 2025. [Online]. Available: <https://developer.arm.com/documentation/ddi0487/latest/>
- [93] X. Chen, Z. Li, T. Jain, V. Narayanan, and A. Burtsev, "Limitations and opportunities of modern hardware isolation mechanisms," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 349–368. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/chen-xiangdong>
- [94] S. Manoni, E. Parisi, R. Tedeschi, D. Rossi, A. Acquaviva, and A. Bartolini, "CVA6-CFI: A first glance at RISC-V control-flow integrity extensions," 2026. [Online]. Available: <https://arxiv.org/abs/2602.04991>
- [95] S. Amar, D. Chisnall, T. Chen, N. W. Filardo, B. Laurie, K. Liu, R. Norton, S. W. Moore, Y. Tao, R. N. M. Watson, and H. Xia, "CHERIOT: Complete memory safety for embedded devices," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 641–653. [Online]. Available: <https://cheriot.org/ibex/flute/architecture/publication/2023/10/30/cheriot-at-micro.html>

- [96] Y. Kim, A. Kar, J. Lee, J. Lee, and H. Kim, "RV-CURE: A RISC-V capability architecture for full memory safety," 2023. [Online]. Available: <https://arxiv.org/abs/2308.02945>
- [97] L. Lamster, M. Unterguggenberger, D. Schrammel, and S. Mangard, "Voodoo: Memory tagging, authenticated encryption, and error correction through MAGIC," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 7159–7176. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/lamster>
- [98] A. Ferraiuolo, A. Baumann, C. Hawblitzel, and B. Parno, "Komodo: Using verification to disentangle secure-enclave hardware from software," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 287–305. [Online]. Available: <https://research.google/pubs/komodo-using-verification-to-disentangle-secure-enclave-hardware-from-software/>
- [99] T. Abera, N. Asokan, L. Davi, J.-E. Ekberg, T. Nyman, A. Paverd, A.-R. Sadeghi, and G. Tsudik, "C-FLAT: Control-flow attestation for embedded systems software," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 743–754. [Online]. Available: <https://research.aalto.fi/en/publications/c-flat-control-flow-attestation-for-embedded-systems-software/>
- [100] G. Dessouky, S. Zeitouni, T. Nyman, A. Paverd, L. Davi, P. Koeberl, N. Asokan, and A.-R. Sadeghi, "LO-FAT: Low-overhead control flow ATtestation in hardware," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017. [Online]. Available: <https://research.aalto.fi/en/publications/lo-fat-low-overhead-control-flow-attestation-in-hardware/>
- [101] I. D. O. Nunes, K. Eldefrawy, N. Rattanavipanon, M. Steiner, and G. Tsudik, "VRASED: A verified Hardware/Software Co-Design for remote attestation," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1429–1446. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/de-oliveira-nunes>
- [102] ARM Limited, "Understanding trace," ARM Ltd., Tech. Rep., 2020.